# Simultaneous Contact Selection and Planning for Contact-Rich Manipulation with Cascaded Optimization

***Abstract*—We propose an optimization-based framework for robust contact-rich manipulation. Recent contact-implicit methods enable online hybrid planning across contact modes, allowing closed-loop manipulation for a given target state and contact location sequence of the robot and object. However, most existing approaches lack the ability to autonomously reason and generate diverse contact location sequences and manipulation trajectories, i.e., active contact location selection, which limits their applicability to relatively simple tasks. Active contact location selection is challenging due to complementarity in contact dynamics and the sparse gradients, making the design of a unified framework for contact selection and planning difficult. To address these challenges, we introduce Simultaneous Contact Selection and Planning (SCSP), a cascaded optimization framework comprising Contact Selection Optimization (CSO) and Contact Planning Optimization (CPO). CSO leverages a surrogate contact model and discrete-continuous optimization to efficiently resolve the nonsmoothness and coupling in contact selection, enabling online search over sampled contact locations. CPO performs prior-guided contact planning by evaluating the reference contact locations produced by CSO and generating corresponding manipulation trajectories in real time for redundant manipulators. Extensive simulations and real-world experiments demonstrate that SCSP produces diverse manipulation behaviors and robust control under inaccurate dynamics and perceptual noise. We further validate the generalization of the framework on challenging manipulation tasks.**

**Project website: https://sites.google.com/view/scsp-robot.**

***Index Terms*—Contact-Rich Manipulation, Contact Selection, Contact-Implicit Trajectory Optimization**

## NOMENCLATURE

| | |
|---|---|
| $p, \boldsymbol{p}$ | A 3D point and its position vector. |
| $\theta, \boldsymbol{\psi}$ | Angle and quaternion vector. |
| $\mathcal{O}, \partial\mathcal{O}$ | Object geometry and surface. |
| $\mathcal{R}, \partial\mathcal{R}$ | Robot geometry and surface. |
| $h$ | Time interval. |
| $\boldsymbol{\lambda}, \boldsymbol{\beta}$ | Contact force vector and contact impulse vector. |
| $\boldsymbol{\phi}$ | Contact distance vector. |
| $\boldsymbol{J}$ | Contact Jacobian. |
| $\boldsymbol{n}, \boldsymbol{t}_i$ | Normal vector and $i$-th tangental vector. |
| $\boldsymbol{x} = [\boldsymbol{p}, \boldsymbol{\psi}]$ | Pose vector consists of position and orientation. |
| $\boldsymbol{q}, \dot{\boldsymbol{q}}, \ddot{\boldsymbol{q}}$ | Robot configuration, velocity and acceleration. |
| $\boldsymbol{s}$ | System state consists of object pose $\boldsymbol{x}_o$ and $\boldsymbol{q}$. |
| $\boldsymbol{v}$ | Velocity vector in Cartesian space. |

## I. INTRODUCTION

CONTACT-Rich manipulation requires robots to intentionally make and break contact with objects to drive them toward desired states. Specifically, the robot needs to plan the contact sequence consisting of approaching trajectory, contact locations, contact forces, and transitions between different contact modes (i.e., contact, separation, sliding, and sticking). Such contact planning is challenging due to the combinatorial complexity of contact modes and the inherent nonsmoothness arising from the hybrid nature of contact dynamics. Early model-based approaches typically relied on hybrid planning, either separating continuous trajectory generation from discrete contact-mode transitions through hierarchical frameworks [1]–[5], or handling the hybrid nature of contact with mixed-integer programs [6]–[8]. However, this separation often suffers from poor scalability caused by the explosion of contact trajectories. Therefore, these methods offer limited manipulation strategies and poor robustness.

Recent advances in contact-rich manipulation have been achieved through both learning-based and model-based methods. Learning-based methods [9]–[17] directly learn mappings from object states to contact strategies, thereby enabling complex contact-rich behaviors without explicitly modeling or optimizing through the hybrid and nonsmooth contact dynamics encountered by model-based methods. However, these methods typically rely on offline training, and their generalization is often demonstrated within specific task families, such as across unseen object geometries or variations of a predefined task. For example, DyWA [17] demonstrates impressive 6D nonprehensile manipulation across diverse object geometries, but remains limited to the specific robot embodiment and task on which it is trained. Consequently, achieving broadly applicable contact-rich manipulation across substantially different robot embodiments and task settings remains challenging.

Contact-implicit trajectory optimization (CITO) subsequently avoids this explicit mode decomposition by modeling robot–object interaction (ROI) with complementary contact dynamics and solving the resulting trajectory optimization problem in a unified framework. CITO was first applied to locomotion [26], [27] and later extended to manipulation. We mainly focus on Contact-Implicit Model Predictive Control (CIMPC), as its receding-horizon formulation continuously replans from the current state and is therefore particularly suitable for real-time closed-loop contact-rich manipulation. CIMPC embeds contact dynamics as constraints in a nonlinear programming (NLP) solver to compute the robot's control inputs online. CIMPC incorporates approximated time-stepping contact dynamics [28], [29] into trajectory optimization, enabling online hybrid contact planning. Because contact models commonly used in physical simulators are often formulated as linear complementarity problems (LCPs) or quadratic programs (QPs), directly using them can hinder

TABLE I
COMPARISON OF CONTACT PLANNING METHODS.

| Methods | Perception | Active Contact Selection | 6D Manip. Exp. | Real-Time | No Obj.-Specific Analytic Geom. |
|---|---|---|---|---|---|
| C3 [18] | Exact State | ✗ | ✗ | ✓ | ✗ |
| CRISP [19] | Exact State | ✗ | ✗ | ✓ | ✗ |
| IDTO [20] | Exact State | ✗ | ✓ | ✓ | ✗ |
| CQDCs [21] | Exact State | ✗ | ✓ | ✓ | ✗ |
| CF-MPC [22] | Exact State | ✗ | ✓ | ✓ | ✓ |
| STOCS-3D [23] | Exact State | ✗ | ✓ | ✗ | ✓ |
| ContactSDF [24] | Vision | ✗ | ✓ | ✓ | ✓ |
| DyWA [17] | Vision | ✗ | ✓ | ✓ | ✓ |
| Sampling [25] | Vision | ✓ | ✓ | ✓ | ✓ |
| **SCSP** | Vision | ✓ | ✓ | ✓ | ✓ |

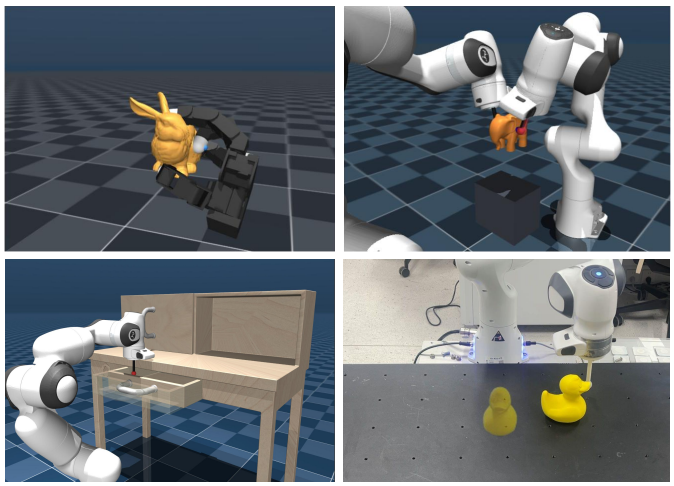


Fig. 1. SCSP enables online contact selection and closed-loop generation of diverse manipulation trajectories, allowing it to address a wide range of challenging contact-rich manipulation tasks.

efficient online optimization. CIMPC methods therefore often adopt approximate contact models that trade physical fidelity for computational efficiency. Representative methods improve the efficiency and scalability of this process through different contact-model approximations and optimization strategies [18], [21], [30]. In particular, CF-MPC [22] introduces a complementarity-free contact model to improve the efficiency of online contact-implicit MPC, substantially accelerating contact-rich optimization.

Although CIMPC enables online planning of contact-mode transitions, actively selecting contact locations on the object remains challenging. Contact geometry and complementarity constraints make the resulting optimization nonsmooth and nonconvex, while contact-free states often provide ineffective gradients toward candidate contacts. Because contact locations are typically determined implicitly by the robot trajectory, short planning horizons and local optimization can obscure the long-term benefits of reaching distant contacts. Recent methods improve contact-rich optimization and execution: Suh et al. [31] introduce a contact-trust-region formulation, Jiang et al. [32] integrate motion–contact planning with tracking, ContactSDF [24] provides differentiable multi-contact models, and CRISP [19] discovers contact sequences within contact-implicit optimization. These methods, however, do not explicitly select task-relevant robot–object contact locations; CRISP is also evaluated primarily in planar settings. Venkatesh et al. [25] address this issue by sampling contact candidates and solving a local CIMPC problem for each, but at the cost of repeated lower-level optimization. Xie et al. [33] use a hierarchical RL–MPC policy to select contacts for a low-level CIMPC controller, but require task-specific training. Efficient task-directed contact selection for closed-loop planning therefore remains unresolved.

In this work, we propose Simultaneous Contact Selection and Planning (SCSP), a cascaded optimization framework for complex contact-rich manipulation. SCSP comprises Contact Selection Optimization (CSO), which performs object-centric search over candidate contact locations and provides contact guidance to steer optimization away from local optima, and Contact Planning Optimization (CPO), which plans robot actions to execute the guided contact while retaining the freedom to realize a nearby feasible contact for flexible execution. A ranking strategy couples the two modules in a cascade by comparing the CSO-guided contact with nearby candidates and passing the resulting reference to CPO.

Table I summarizes the scope of SCSP relative to representative contact-planning methods. The contributions are:

- An efficient contact selection method, CSO, is developed for online contact-guidance generation. It samples a candidate point set and performs batched optimization to avoid direct surface-gradient search and the associated nonsmoothness of contact selection. A lightweight surrogate contact model (SCM) enables fast evaluation of candidate utility and selection of the best contact as guidance.
- A contact planning method, CPO, is proposed with a unified cost formulation for generating approach, contact-establishment, and contact-switching trajectories.
- A ranking strategy forms the cascade between contact guidance and planning by ranking the guided contact together with nearby feasible candidates. This preserves task-directed guidance while allowing CPO to select an executable nearby contact, thereby maintaining execution flexibility and contact-trajectory diversity.
- Extensive simulation and real-world experiments validate SCSP across diverse object geometries and contact-rich

tasks. Real-world experiments further demonstrate robust vision-based nonprehensile manipulation using only simple contact calibration and without exact object perception.

The rest of the paper is organized as follows. Section II reviews the preliminaries. The CSO and CPO are introduced in Sections III and IV, respectively. Section V presents the experimental results, and Section VII evaluates structured extensions of the SCSP architecture to additional contact-rich manipulation settings. Finally, discussion and limitations are presented in Section VIII, and conclusions are drawn in Section IX.

## II. Preliminaries

We begin with a brief overview of the contact-dynamics and contact-planning formulations used throughout this work. Section II-A introduces time-stepping contact dynamics and the associated complementarity formulation. Section II-B introduces model-based contact planning and reviews CIMPC as a representative online formulation. Section II-C reviews the complementarity-free contact model that motivates the lightweight contact-model approximation used later in CSO.

### A. Contact Dynamics

Contact dynamics provides a physical description of interactions among the robot, object, and environment. Here, we consider the commonly used Anitescu model [29], which is formulated as the following time-stepping equation:

$$\epsilon \boldsymbol{M}_o \boldsymbol{v}_o = h\boldsymbol{\tau}_o + \sum_{i=1}^{n_c} \boldsymbol{J}_{o,i}^\top \boldsymbol{\beta}_i, \tag{1a}$$

$$h\boldsymbol{K}_r\left(h\boldsymbol{v}_r - \boldsymbol{u}\right) = h\boldsymbol{\tau}_r + \sum_{i=1}^{n_c} \boldsymbol{J}_{r,i}^\top \boldsymbol{\beta}_i. \tag{1b}$$

Here, $\boldsymbol{\beta}_i$ denotes the contact impulse over the time interval $h$. In the contact-force formulation below, we use $\boldsymbol{\lambda}_i$ to denote the corresponding contact force. Assuming that the contact force is approximately constant within one time step, they are related by $\boldsymbol{\beta}_i \approx h\boldsymbol{\lambda}_i$. $\epsilon\boldsymbol{M}_o$, $\boldsymbol{v}_o$, $\boldsymbol{\tau}_o$, and $\boldsymbol{J}_{o,i}$ are the regularized mass matrix, velocity change, non-contact force, and contact Jacobian of the object, respectively. $\boldsymbol{K}_r$, $\boldsymbol{v}_r$, $\boldsymbol{\tau}_r$, and $\boldsymbol{J}_{r,i}$ are the robot stiffness matrix, velocity, non-contact torque, and contact Jacobian, respectively. The control input is defined as $\boldsymbol{u} = \Delta\boldsymbol{q}$.

The above equations describe, from the perspective of momentum balance, how the robot influences the object's state through contact, as well as how the object interacts with the environment. Here, we follow [22], [34] and consider the robot to be under impedance control [35], modeling the robot–object interaction as a spring system. The contact constraints between contact forces and system motion are typically formulated as the following cone complementarity problem [36]:

$$\mathcal{K}_i \ni \boldsymbol{\lambda}_i \perp \boldsymbol{J}_i\boldsymbol{v} + \frac{1}{h}\begin{bmatrix}\phi_i\\0\\0\end{bmatrix} \in \mathcal{K}_i^*, \qquad \forall i \in \{1,\dots,n_c\}, \tag{2}$$

where $\mathcal{K}_i = \left\{\boldsymbol{\lambda}_i \in \mathbb{R}^3 \,\middle|\, \mu_i\lambda_i^n \geq \left\|\boldsymbol{\lambda}_i^d\right\|\right\}$ is the Coulomb friction cone, and $\mathcal{K}_i^*$ is the dual cone as $\mathcal{K}_i^* = \left\{\boldsymbol{v} \,\middle|\, \boldsymbol{J}_i^n\boldsymbol{v} + \frac{\phi_i}{h} \geq \mu_i \left\|\boldsymbol{J}_i^d\boldsymbol{v}\right\|\right\}$.

Equations (1) and (2) are the Karush–Kuhn–Tucker (KKT) [37] optimality conditions of the following primal optimization [29]:

$$\min_{\boldsymbol{v}} \quad \frac{1}{2}h^2\boldsymbol{v}^\top\boldsymbol{Q}\boldsymbol{v} - h\boldsymbol{v}^\top\boldsymbol{b}(\boldsymbol{u}), \tag{3a}$$

$$\text{s.t.} \quad \boldsymbol{J}_i\boldsymbol{v} + \frac{1}{h}\begin{bmatrix}\phi_i\\0\\0\end{bmatrix} \in \mathcal{K}_i^*, \qquad i \in \{1,\dots,n_c\}. \tag{3b}$$

where $\boldsymbol{Q} \in \mathbb{R}^{(n_o+n_{\mathrm{DoF}})\times(n_o+n_{\mathrm{DoF}})}$ and $\boldsymbol{b}(\boldsymbol{u}) \in \mathbb{R}^{n_o+n_{\mathrm{DoF}}}$ are

$$\boldsymbol{Q} := \begin{bmatrix}\epsilon\boldsymbol{M}_o/h^2 & \boldsymbol{0}\\ \boldsymbol{0} & \boldsymbol{K}_r\end{bmatrix}, \qquad \boldsymbol{b}(\boldsymbol{u}) := \begin{bmatrix}\boldsymbol{\tau}_o\\ \boldsymbol{K}_r\boldsymbol{u} + \boldsymbol{\tau}_r\end{bmatrix}. \tag{4}$$

Here, $n_o$ and $n_{\mathrm{DoF}}$ are the dimensions of the object and robot states, respectively, and $n_c$ is the number of contacts.

Problem (3) is a second-order cone program (SOCP) and is often approximated by a QP through a polyhedral approximation of the friction cone:

$$\left(\boldsymbol{J}_i^n - \mu_i\boldsymbol{J}_{i,j}^d\right)\boldsymbol{v} + \frac{\phi_i}{h} \geq 0, \quad i \in \{1,\dots,n_c\}, \quad j \in \{1,\dots,n_d\}, \tag{5}$$

where $n_d$ is the number of friction-cone facets. Solving (3) corresponds to one step of physically consistent contact simulation. The resulting velocity $\boldsymbol{v}$ can then be used to integrate the current state as $\boldsymbol{q}_{t+1} = \boldsymbol{q}_t \oplus h\boldsymbol{v}$. Problem (3) and its variants are widely used in physics simulators.

### B. Model-based Contact Planning

Contact planning refers to generating trajectories for approaching, making contact, and breaking contact with the object to manipulate it to a target state. Model-based contact planning methods integrate contact dynamics and solve trajectory optimization problems. As a representative contact planning method, CIMPC is formulated as the following optimization problem:

$$\begin{aligned}\min_{\boldsymbol{u}_{0:N-1}} \quad & \sum_{k=0}^{N-1} \ell(\boldsymbol{x}_k, \boldsymbol{u}_k) + V_f(\boldsymbol{x}_N)\\ \text{s.t.} \quad & \boldsymbol{x}_{k+1} = f(\boldsymbol{x}_k, \boldsymbol{u}_k), \quad k = 0,\dots,N-1,\\ & \boldsymbol{x}_k \in \mathcal{X}, \quad k = 0,\dots,N,\\ & \boldsymbol{u}_k \in \mathcal{U}, \quad k = 0,\dots,N-1.\end{aligned} \tag{6}$$

in which $\boldsymbol{x}_k$ is the system state, including the object pose $q_k$ and robot configuration, and $f$ denotes the contact dynamics. The stage cost $\ell$ can take various forms and is typically used to guide the process of approaching the contact location. The terminal cost $V_f$ is usually defined as the error between the object pose and the desired pose:

$$\ell_{\mathrm{pose}} = w_{\mathrm{pos}}\left\|\boldsymbol{p}_{\mathrm{obj}} - \boldsymbol{p}_{\mathrm{goal}}\right\|_2^2 + w_{\mathrm{quat}}\left(1 - \left(\boldsymbol{\psi}_{\mathrm{obj}}^\top\boldsymbol{\psi}_{\mathrm{goal}}\right)^2\right). \tag{7}$$

The hybrid nature of contact dynamics renders (6) a piecewise-smooth nonlinear complementarity problem (NCP).

Directly using the full contact model in (3) within receding-horizon trajectory optimization remains computationally demanding, motivating the efficient contact-model approximations introduced below.

### C. Complementarity-free Model

We next introduce the complementarity-free contact model proposed in [22], which motivates the surrogate contact-model approximation used later in CSO. Problem (3) has the dual form:

$$\max_{\boldsymbol{\beta}\geq\mathbf{0}} \quad -\frac{1}{2h^2}\boldsymbol{\beta}^\top\left(\widetilde{\boldsymbol{J}}\boldsymbol{Q}^{-1}\widetilde{\boldsymbol{J}}^\top+\boldsymbol{R}\right)\boldsymbol{\beta} - \frac{1}{h}\left(\widetilde{\boldsymbol{J}}\boldsymbol{Q}^{-1}\boldsymbol{b}+\widetilde{\boldsymbol{\phi}}\right)^\top\boldsymbol{\beta}-\frac{1}{2}\boldsymbol{b}^\top\boldsymbol{Q}^{-1}\boldsymbol{b}. \quad (8)$$

in which $\boldsymbol{R}$ is a small regularization term commonly used to enhance numerical stability, and

$$\widetilde{\boldsymbol{J}}=\begin{bmatrix}\boldsymbol{J}_1^n-\mu_1\boldsymbol{J}_{1,1}^t\\ \vdots\\ \boldsymbol{J}_1^n-\mu_1\boldsymbol{J}_{1,n_t}^t\\ \vdots\\ \boldsymbol{J}_{n_c}^n-\mu_{n_c}\boldsymbol{J}_{n_c,1}^t\\ \vdots\\ \boldsymbol{J}_{n_c}^n-\mu_{n_c}\boldsymbol{J}_{n_c,n_t}^t\end{bmatrix},\widetilde{\boldsymbol{\phi}}=\begin{bmatrix}\phi_1\\ \vdots\\ \phi_1\\ \vdots\\ \phi_{n_c}\\ \vdots\\ \phi_{n_c}\end{bmatrix},\boldsymbol{\beta}=\begin{bmatrix}\beta_{1,1}\\ \vdots\\ \beta_{1,n_d}\\ \vdots\\ \beta_{n_c,1}\\ \vdots\\ \beta_{n_c,n_d}\end{bmatrix}. \quad (9)$$

Here, $\boldsymbol{\beta}$ is not only a dual variable but also has a physical interpretation as the contact impulse. Specifically, the solution of (8) satisfies the following complementarity constraints:

$$\mathbf{0}\leq\boldsymbol{\beta}\perp\frac{1}{h}\left(\widetilde{\boldsymbol{J}}\boldsymbol{Q}^{-1}\widetilde{\boldsymbol{J}}^\top+\boldsymbol{R}\right)\boldsymbol{\beta}+\left(\widetilde{\boldsymbol{J}}\boldsymbol{Q}^{-1}\boldsymbol{b}+\widetilde{\boldsymbol{\phi}}\right)\geq\mathbf{0}. \quad (10)$$

Following [22], we replace the dense inverse $(\widetilde{\boldsymbol{J}}\boldsymbol{Q}^{-1}\widetilde{\boldsymbol{J}}^\top+\boldsymbol{R})^{-1}$ by a positive-definite diagonal matrix $\boldsymbol{K}(\boldsymbol{q})$ chosen as a local approximation. This replacement defines a modified dual problem; it is not an equality that holds for a general multi-contact geometry. With $\boldsymbol{D}=\boldsymbol{K}^{-1}$, the modified problem has the following closed-form solution:

$$\boldsymbol{\beta}^+=\max\left(-h\boldsymbol{K}(\boldsymbol{q})\left(\widetilde{\boldsymbol{J}}\boldsymbol{Q}^{-1}\boldsymbol{b}+\widetilde{\boldsymbol{\phi}}\right),\mathbf{0}\right). \quad (11)$$

The complementarity-free model is then written as:

$$f_{\text{cf}}:\begin{cases}\boldsymbol{v}^+=\frac{1}{h}\boldsymbol{Q}^{-1}\boldsymbol{b}+\frac{1}{h}\boldsymbol{Q}^{-1}\widetilde{\boldsymbol{J}}^\top\boldsymbol{\beta}^+,\\ \boldsymbol{x}_{k+1}=\boldsymbol{x}_k+h\left(\boldsymbol{v}_k+\boldsymbol{v}^+\right).\end{cases} \quad (12)$$

where $\boldsymbol{\beta}$ denotes the contact impulses of all contact points and $\boldsymbol{v}^+$ combines the object velocity $\boldsymbol{v}_o$ and robot velocity $\boldsymbol{v}_r$.

The modified problem is exact for the chosen diagonal surrogate, while its discrepancy from the original contact model depends on the discarded off-diagonal coupling. The predictive fidelity of SCM is evaluated empirically in Section V.

## III. Contact Selection Optimization

In this section, we present the detailed design of CSO. We begin by analyzing the key challenges of integrating contact selection into CIMPC, which limit the diversity of existing methods. We then formulate the contact selection problem and discuss the main difficulties in solving it. Finally, we present the detailed design of our CSO module for online contact selection.

### A. Why Contact Selection is necessary?

Contact selection refers to choosing object contact locations $\{\boldsymbol{p}_{\mathcal{O},i}\}\subset\partial\mathcal{O}$ that can generate task-beneficial wrenches for changing the object pose. According to (1a) and (1b), the object wrench is jointly determined by the contact Jacobian, which depends on the contact location and its local geometry, and the contact impulse $\boldsymbol{\beta}_i$ (or, equivalently, the contact force $\boldsymbol{\lambda}_i$). For some contact locations, no force within the corresponding friction cone can produce a wrench that improves the object pose; contact switching is therefore necessary.

CIMPC models robot–object interaction (ROI) with contact dynamics whose informative gradients are confined to the low-dimensional contact manifold $\mathcal{S}_{\text{contact}}$ embedded in the full state space $\mathcal{S}$. The contact-free manifold $\mathcal{S}_{\text{free}}$ provides no effective contact-induced gradient, making autonomous contact selection difficult and requiring attractive costs or reference trajectories to induce contact. Although CIMPC can choose between contact and non-contact modes, its short receding horizon may not reveal, within the fixed horizon, the benefit of reaching a distant contact location.

Rather than integrating contact selection into the CIMPC formulation, we study how to formulate contact selection as a separate optimization problem. As illustrated in Fig. 2, we first simplify the problem by ignoring the planning of the robot contact locations $\{\boldsymbol{p}_{\mathcal{R},i}\}$ and thereby obtain the following object-centric optimization problem:

$$\begin{aligned}\min_{\boldsymbol{p}_{\text{r}}\in\partial\mathcal{O},\boldsymbol{\lambda}\in\mathcal{K}}\ &\ell_{\text{cso}}(\boldsymbol{x}_T)\\ \text{s.t.}\quad &\boldsymbol{x}_{k+1}=f(\boldsymbol{x}_k,\boldsymbol{p}_{\text{r}},\boldsymbol{\lambda}),\quad k=0,\dots,T-1.\end{aligned} \quad (13)$$

where $\mathcal{K}$ is the set of feasible contact forces and $\ell_{\text{cso}}(\cdot)$ is the objective of contact selection. $\boldsymbol{p}_{\text{r}}\in\mathbb{R}^{3\times n_{\text{r}}}$ represents the locations where the robot applies contacts to the object, with $n_{\text{r}}$ denotes the number of robot contacts. (13) addresses the gradient sparsity issue arising from hybrid planning in the original contact-rich manipulation problem by defining CSO to solve for low-cost sampled contact locations and contact forces on $\mathcal{S}_{\text{contact}}$. The neglected process of the robot approaching $\mathcal{S}_{\text{contact}}$ from its current state is the primary source of the gradient sparsity. Note that (13) is hard to solve due to the coupling of contact location $\boldsymbol{p}_{\text{r}}$ and the contact force $\boldsymbol{\lambda}=[\boldsymbol{\lambda}_{\text{env}},\boldsymbol{\lambda}_{\text{r}}]$. Therefore, we adopt a commonly used simplification for handling nonconvex optimization problems to reformulate (13) as subproblems:

$$\min_{\boldsymbol{p}_{\text{r}}\in\partial\mathcal{O}}\quad V(\boldsymbol{p}_{\text{r}}), \quad (14)$$

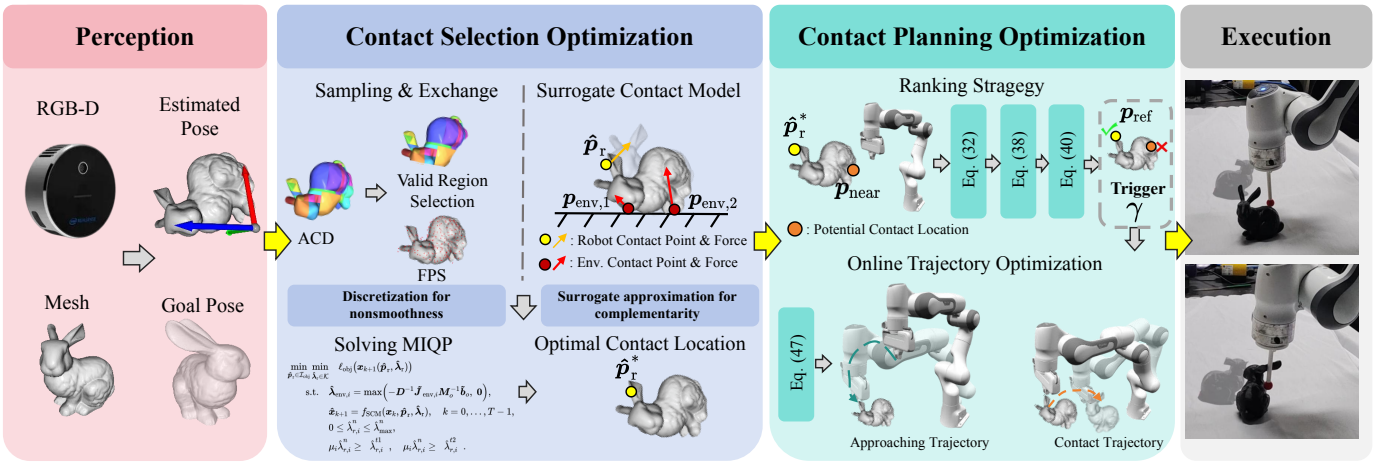



Fig. 2. Framework structure of SCSP. SCSP consists of Contact Selection Optimization (CSO) and Contact Planning Optimization (CPO). CSO discretizes nonsmooth geometry through the Sampling & Exchange method and introduces a surrogate contact model for efficient approximation of contact dynamics. CSO online computes the optimal contact location among candidate contact set $\hat{\boldsymbol{p}}_{\mathrm{r}}^{*}$ as contact prior. The ranking strategy in CPO further evaluates whether the nearest point $\boldsymbol{p}_{\mathrm{near}}$ can serve as a feasible suboptimal substitute for $\hat{\boldsymbol{p}}_{\mathrm{r}}^{*}$ as the reference contact location, after which trajectory optimization is solved online to generate the approach and contact trajectories.

in which:

$$V(\boldsymbol{p}_{\mathrm{r}}) := \min_{\boldsymbol{\lambda}_{\mathrm{r}} \in \mathcal{K}} \quad \ell_{\mathrm{cso}}(\boldsymbol{x}_T) \\ \text{s.t.} \quad \boldsymbol{x}_{k+1} = f(\boldsymbol{x}_k, \boldsymbol{p}_{\mathrm{r}}, \boldsymbol{\lambda}_{\mathrm{r}}, \boldsymbol{\lambda}_{\mathrm{env}}), \quad k = 0, \ldots, T-1 \tag{15}$$

The physical interpretation of (14) and (15) is that a contact location is regarded as optimal if applying the best possible combination of contact force $\boldsymbol{\lambda}^{*}$ yields the minimum motion cost of the object. Despite the simplifications, (14) remains a complex problem with nonsmooth constraints $\boldsymbol{p}_{\mathrm{r}} \in \partial\mathcal{O}$ that depend on the object geometry. Meanwhile, (15) is a complementary problem with coupled environment forces $\boldsymbol{\lambda}_{\mathrm{env}}$ and robot contact forces $\boldsymbol{\lambda}_{\mathrm{r}}$. The nonconvexity makes online CSO intractable. We next present the solutions to further simplify both (14) and (15) to improve computational efficiency.

### B. Sampling and Exchange Method

The nonsmoothness on the object geometry leads to discontinuities in the gradient of (14). The inherently nonsmooth nature of the problem motivates us to adopt a sampling-based method. To simplify (14), we uniformly sample a set of candidate contact point set $\mathcal{I}_{\mathrm{obj}} = \{\boldsymbol{p}_i\}_{i=1}^{n_s}$ on the object surface $\partial\mathcal{O}$, thus reformulate (14) as:

$$\min_{\boldsymbol{p}_{\mathrm{r}} \in \mathcal{I}_{\mathrm{obj}}} \quad V(\boldsymbol{p}_{\mathrm{r}}), \tag{16}$$

When the sampling is sufficiently uniform, (16) provides a good approximation to (14) and is not affected by discontinuous gradients caused by the geometry. To obtain a high quality candidate set, $\mathcal{I}_{\mathrm{obj}}$ should satisfy the following conditions:

- **Physical reachability**, which requires that candidate contact locations be reachable by the robot, i.e., within the robot workspace and not occluded.
- **Contact stability**, which requires avoiding regions with high curvature where stable contact is difficult to apply.
- **Uniform distribution**, which means the sampled points are representative of approximating the original geometry.

These conditions require the sampling method to maintain uniform coverage while prioritizing manipulability over local geometric details. To address this, we propose an efficient sampling strategy as shown in Algorithm 1. We first perform approximate convex decomposition (ACD) [38] to obtain a convex approximation $\mathcal{F}$ of the original geometry $\mathcal{O}$. ACD decomposes the object into convex polygons, mitigating interference caused by local geometric details and reducing computational cost. Moreover, these polygons are typically larger and flatter, which implies greater contact stability and improved reachability.

Then we perform Farthest-Point Sampling (FPS) on the convex approximation to uniformly sample points on the polygons. Starting from an arbitrary point as a sample, the algorithm iteratively selects the points that maximize the minimal Euclidean distance to all previously chosen samples:

$$p'_{k+1} = \arg\max_{p_i \in \mathcal{F}} \min_{p'_j \in \mathcal{I}_k} \|\boldsymbol{p}_i - \boldsymbol{p}'_j\|^2, \tag{17}$$

where $\mathcal{I}_k$ is the set of already selected samples. The sampling process is performed offline. Solving (17) yields a uniformly distributed set of points $\mathcal{I}_{\mathrm{obj}}$. For each sampled point $p'_j \in \mathcal{I}_k$, we also construct its local coordinate frame to obtain the tangential and normal vectors as local contact information. The local orthonormal frame is constructed as $(\boldsymbol{n}_i, \boldsymbol{t}_{1,i}, \boldsymbol{t}_{2,i})$ in which the normal vector $\boldsymbol{n}_i$ is obtained from the mesh or point-cloud normal estimate, and two orthogonal tangent directions are computed as:

$$\boldsymbol{t}_{1,i} = \frac{\boldsymbol{a} - (\boldsymbol{a} \cdot \boldsymbol{n}_i)\boldsymbol{n}_i}{\|\boldsymbol{a} - (\boldsymbol{a} \cdot \boldsymbol{n}_i)\boldsymbol{n}_i\|}, \ \boldsymbol{t}_{2,i} = \boldsymbol{n}_i \times \boldsymbol{t}_{1,i}, \tag{18}$$

where $\boldsymbol{a}$ is an arbitrary fixed axis. When the reference axis $a$ is nearly parallel to the surface normal $n_i$, we select another coordinate axis as the reference axis to avoid numerical singularity. We use this local geometric information to compute the Jacobian required for contact force optimization. Furthermore, a KD-tree $\mathcal{T} = \{(\boldsymbol{p}_i, \boldsymbol{n}_i, \boldsymbol{t}_{1,i}, \boldsymbol{t}_{2,i})\}_{i=1}^{n_s}$ is constructed over the set $\mathcal{I}_{\mathrm{obj}}$ to enable efficient nearest-neighbor queries.

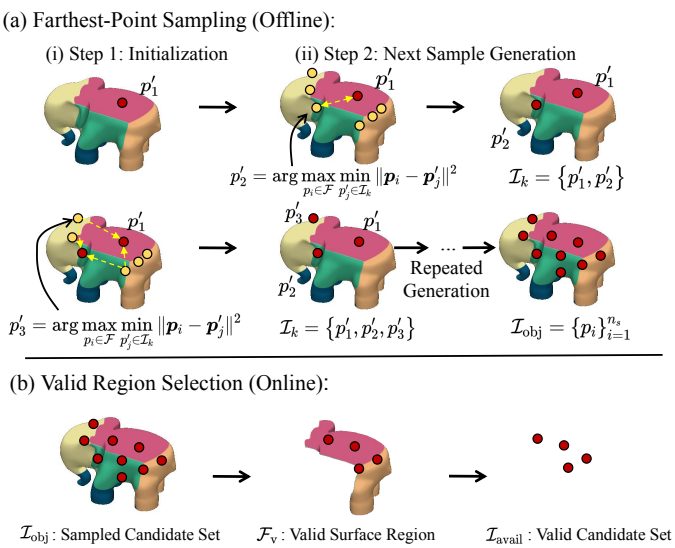



Fig. 3. The sampling strategy in CSO. The candidate point set is sampled sequentially through: (a) Farthest-Point Sampling to uniformly sample points on the object, and (b) Valid Region Selection to select valid points for contact.

**Algorithm 1** Sampling and Exchange Method

**Input:** Object geometry $\mathcal{O}$, sample count $K$, fixed axis $\boldsymbol{a}$
**Output:** $\mathcal{T} = \{(\boldsymbol{p}_i, \boldsymbol{n}_i, \boldsymbol{t}_{1,i}, \boldsymbol{t}_{2,i})\}_{i=1}^{n_s}$ and $\mathcal{I}_{\text{avail}}$.

$\mathcal{F} \leftarrow \text{ACD}(\mathcal{O})$ ▷ Convex approximation
Initialize $\mathcal{I}_k \leftarrow \{p_1'\}$ with an arbitrary seed in $\mathcal{F}$
**for** $k = 1$ to $K - 1$ **do**
  $p_{k+1}' \leftarrow \arg\max_{p_i' \in \mathcal{F}} \min_{p_j \in \mathcal{I}_k} \|\boldsymbol{p}_i' - \boldsymbol{p}_j'\|^2$
  $\mathcal{I}_{\text{obj}} \leftarrow \mathcal{I}_k \cup \{p_{k+1}'\}$ ▷ Obtain candidate set $\mathcal{I}_{\text{obj}}$
**end for**
$\mathcal{F}_{\text{v}} \leftarrow \text{getValidRegion}(\mathcal{F})$ ▷ Task-specific segment
**for all** $p_i \in \mathcal{I}_{\text{obj}}$ **do**
  **if** $p_i \notin \mathcal{F}_{\text{v}}$ **then**
    Remove $p_i$ from $\mathcal{I}_{\text{obj}}$ ▷ Valid region selection
    **continue**
  **end if** ▷ Obtain $\mathcal{I}_{\text{avail}}$
  Estimate normal $\boldsymbol{n}_i$ from $\mathcal{F}$ at $p_i$
  $\boldsymbol{t}_{1,i} \leftarrow \dfrac{\boldsymbol{a} - (\boldsymbol{a} \cdot \boldsymbol{n}_i)\boldsymbol{n}_i}{\|\boldsymbol{a} - (\boldsymbol{a} \cdot \boldsymbol{n}_i)\boldsymbol{n}_i\|}$, $\boldsymbol{t}_{2,i} \leftarrow \boldsymbol{n}_i \times \boldsymbol{t}_{1,i}$
**end for**
**return** $\mathcal{T} = \{(\boldsymbol{p}_i, \boldsymbol{n}_i, \boldsymbol{t}_{1,i}, \boldsymbol{t}_{2,i})\}_{i=1}^{n_s}$

An online Valid Region Selection procedure is further applied to identify feasible and task-relevant surface regions, filtering out unreachable or low-quality points such as occluded regions, regions outside the robot workspace, and regions that violate task-specific semantic masks (for example, handles or forbidden surfaces). The valid region is maintained by applying a mask to $\mathcal{F}$ to obtain $\mathcal{F}_{\text{v}}$, and then selecting points on $\mathcal{F}_{\text{v}}$ to obtain $\mathcal{I}_{\text{avail}}$. To better illustrate the process, consider the toy example shown in Fig. 3. First, following the procedure illustrated in Fig. 3(a), we iteratively apply (17) to obtain a set of uniformly distributed sampling points over the processed object mesh. We then partition the surface online into a set of valid regions and masked regions, and select the points that lie within the valid regions to form the candidate set $\mathcal{I}_{\text{avail}}$, as shown in Fig. 3(b).

### C. Surrogate Contact Model

The sampling strategy transforms the original optimization into a discrete–continuous optimization problem. We further simplify (15) to improve efficiency. Most existing contact models are computationally expensive for scenarios that require repeatedly solving (15). Moreover, the complementarity in contact dynamics and the coupling between $\boldsymbol{\lambda}_{\text{env}}$ and $\boldsymbol{\lambda}_{\text{r}}$ make the optimization difficult to converge, leading to physically inconsistent results. To address this issue and enable online contact selection, we present a surrogate contact model (SCM) as a local approximation of the original contact dynamics. The procedure of SCM is illustrated in Fig. 4. The approximation relies on the following assumptions of the object-environment contact:

**Assumption 1** (Fixed Contact Set)**.** The object–environment contact location remains unchanged in each SCM rollout, i.e., no new contacts are created and existing contacts remain.

**Assumption 2** (Multi-Contact Decoupling)**.** The contact between the object and the environment can be approximated by independent spring–damper systems, neglecting higher-order dynamic couplings between different contact locations.

We reformulate the original contact dynamics in (1a) as:

$$\epsilon \boldsymbol{M}_o \boldsymbol{v}_o = h(\boldsymbol{\tau}_o + \sum\nolimits_{i=1}^{n_{\text{r}}} \boldsymbol{J}_{\text{r},i}^{\top} \boldsymbol{\lambda}_{\text{r},i} + \sum\nolimits_{j=1}^{n_{\text{env}}} \boldsymbol{J}_{\text{env},j}^{\top} \boldsymbol{\lambda}_{\text{env},j}), \quad (19)$$

and we denote $\tilde{\boldsymbol{b}}_o = h(\boldsymbol{\tau}_o + \sum_{i=1}^{n_{\text{r}}} \boldsymbol{J}_{\text{r},i}^{\top} \boldsymbol{\lambda}_{\text{r},i})$. Similar to (8), the physically consistent environment contact force can be obtained by solving the following dual problem:

$$\begin{aligned} \max_{\boldsymbol{\lambda}_{\text{env}} \geq \boldsymbol{0}} \quad & -\frac{1}{2} \boldsymbol{\lambda}_{\text{env}}^{\top} \left( \tilde{\boldsymbol{J}}_{\text{env}} \boldsymbol{M}_o^{-1} \tilde{\boldsymbol{J}}_{\text{env}}^{\top} + \boldsymbol{R} \right) \boldsymbol{\lambda}_{\text{env}} \\ & - (\tilde{\boldsymbol{J}}_{\text{env}} \boldsymbol{M}_o^{-1} \tilde{\boldsymbol{b}}_o + \tilde{\boldsymbol{\phi}})^{\top} \boldsymbol{\lambda}_{\text{env}} - \frac{1}{2} \tilde{\boldsymbol{b}}_o^{\top} \boldsymbol{M}_o^{-1} \tilde{\boldsymbol{b}}_o, \end{aligned} \quad (20)$$

and the solution satisfies the complementarity condition

$$\boldsymbol{0} \leq \boldsymbol{\lambda}_{\text{env}} \perp \boldsymbol{W}_{\text{env}} \boldsymbol{\lambda}_{\text{env}} + \boldsymbol{g}_{\text{env}} \geq \boldsymbol{0}. \quad (21)$$

where

$$\boldsymbol{W}_{\text{env}} = \left( \tilde{\boldsymbol{J}}_{\text{env}} \boldsymbol{M}_o^{-1} \tilde{\boldsymbol{J}}_{\text{env}}^{\top} + \boldsymbol{R} \right), \ \boldsymbol{g}_{\text{env}} = \left( \tilde{\boldsymbol{J}}_{\text{env}} \boldsymbol{M}_o^{-1} \tilde{\boldsymbol{b}}_o + \tilde{\boldsymbol{\phi}} \right). \quad (22)$$

Here $\boldsymbol{W}_{\text{env}}$ is the Delassus matrix associated with object–environment contact. Since the contact force at each contact point is decomposed into one normal and two tangential components $\boldsymbol{\lambda}_i = [\lambda_i^n, \lambda_i^{t_1}, \lambda_i^{t_2}]^{\top} \in \mathbb{R}^3$, the Delassus matrix $\boldsymbol{W}_{\text{env}}$ admits a block structure

$$\boldsymbol{W}_{\text{env}} = \begin{bmatrix} \boldsymbol{W}_{11} & \boldsymbol{W}_{12} & \cdots & \boldsymbol{W}_{1 n_{\text{env}}} \\ \boldsymbol{W}_{21} & \boldsymbol{W}_{22} & \cdots & \boldsymbol{W}_{2 n_{\text{env}}} \\ \vdots & \vdots & \ddots & \vdots \\ \boldsymbol{W}_{n_{\text{env}} 1} & \boldsymbol{W}_{n_{\text{env}} 2} & \cdots & \boldsymbol{W}_{n_{\text{env}} n_{\text{env}}} \end{bmatrix},$$

where each block $\boldsymbol{W}_{ij} \in \mathbb{R}^{3 \times 3}$ is

$$\boldsymbol{W}_{ij} = \begin{bmatrix} \boldsymbol{J}_i^n \\ \boldsymbol{J}_i^{t_1} \\ \boldsymbol{J}_i^{t_2} \end{bmatrix} \boldsymbol{M}_o^{-1} \begin{bmatrix} (\boldsymbol{J}_j^n)^{\top} & (\boldsymbol{J}_j^{t_1})^{\top} & (\boldsymbol{J}_j^{t_2})^{\top} \end{bmatrix} + \boldsymbol{R}_{ij}.$$

The diagonal blocks $\boldsymbol{W}_{ii}$ describe the local effective inertia at each contact, indicating how forces applied at contact $i$ affect motion along its own constraint directions. The off-diagonal blocks $\boldsymbol{W}_{ij}$ $(i \neq j)$ represent dynamic coupling between different contacts. Under Assumption 2, these inter-contact couplings are negligible. Therefore, $\boldsymbol{W}_{\text{env}}$ can be approximated by a block-diagonal matrix:

$$\boldsymbol{W}_{\text{env}} = \text{blockdiag}(\boldsymbol{W}_{11}, \boldsymbol{W}_{22}, \ldots, \boldsymbol{W}_{n_{\text{env}} n_{\text{env}}}), \tag{23}$$

and (21) is decomposed to subproblems:

$$\mathbf{0} \leq \boldsymbol{\lambda}_{\text{env},i} \perp \boldsymbol{W}_{ii}\boldsymbol{\lambda}_{\text{env},i} + (\tilde{\boldsymbol{J}}_{\text{env},i}\boldsymbol{M}_o^{-1}\tilde{\boldsymbol{b}}_o + \tilde{\boldsymbol{\phi}}_i) \geq \mathbf{0}. \tag{24}$$

Furthermore, if $\boldsymbol{W}_{ii}$ admits the low-rank decomposition:

$$\boldsymbol{W}_{ii} = \boldsymbol{D} + \boldsymbol{E}, \tag{25}$$

where $\boldsymbol{D}$ is a diagonal matrix and $\boldsymbol{E}$ is relatively small, (24) has the approximated closed-form solution:

$$\hat{\boldsymbol{\lambda}}_{\text{env},i} = \max\left(-\boldsymbol{D}^{-1}(\tilde{\boldsymbol{J}}_{\text{env},i}\boldsymbol{M}_o^{-1}\tilde{\boldsymbol{b}}_o + \tilde{\boldsymbol{\phi}}_i),\ \mathbf{0}\right). \tag{26}$$

With (26), we can obtain the following lemma:

**Lemma 1.** Under Assumption 2, the environment contact force $\boldsymbol{\lambda}_{\text{env}}$ is an explicit function of $\boldsymbol{\lambda}_{\text{r}}$.

To obtain a high-quality approximation, we decompose $\boldsymbol{W}_{ii}$ in (25) as:

$$\boldsymbol{D} = \text{diag}(\boldsymbol{W}_{ii}) + \epsilon\boldsymbol{I}, \tag{27}$$

where $\epsilon$ is a small regularization coefficient to ensure numerical stability. (27) gives the closest diagonal approximation to $\boldsymbol{W}_{ii}$ in the Frobenius norm[1]. However, it should be noted that (26) is incompatible with gradient-based solvers. When the contact set changes within the CSO horizon, variations in $\tilde{\boldsymbol{J}}_{\text{env},i}$ and $\tilde{\boldsymbol{\phi}}_i$ introduce additional non-smoothness. To address this, we leverage Assumption 1 to further derive:

**Lemma 2.** The environment contact force $\boldsymbol{\lambda}_{\text{env}}$ is a piecewise linear function of $\boldsymbol{\lambda}_{\text{r}}$:

$$\hat{\boldsymbol{\lambda}}_{\text{env},i} = \max\left(-\boldsymbol{D}^{-1}\tilde{\boldsymbol{J}}_{\text{env},i}\boldsymbol{M}_o^{-1}(\boldsymbol{\tau}_o + \sum\nolimits_{i=1}^{n_{\text{r}}} \boldsymbol{J}_{\text{r},i}^{\top}\hat{\boldsymbol{\lambda}}_{\text{r},i}),\ \mathbf{0}\right). \tag{28}$$

*Proof.* Based on Assumption 1, we can obtain that $\phi \equiv 0$, leading to (28). Moreover, since the contact geometry set remains invariant over the time interval, $\tilde{\boldsymbol{J}}_{\text{env},i}$ is a constant matrix, thus $\boldsymbol{W}_{ii}$ is constant, and $\boldsymbol{D}$, $\boldsymbol{M}_o^{-1}$ are all constant matrices. Therefore, (28) is piecewise linear.

Substituting (28) into (19), we obtain a lightweight explicit model:

$$f_{\text{SCM}}: \begin{cases} \hat{\boldsymbol{v}}^{+} = \frac{1}{h}\boldsymbol{M}_o^{-1}\tilde{\boldsymbol{b}}_o + \frac{1}{h}\boldsymbol{M}_o^{-1}\sum_{i=1}^{n_{\text{env}}} \tilde{\boldsymbol{J}}_{\text{env},i}^{\top}\hat{\boldsymbol{\lambda}}_{\text{env},i}, \\ \hat{\boldsymbol{x}}_{k+1} = \boldsymbol{x}_k + h(\boldsymbol{v}_k + \hat{\boldsymbol{v}}^{+}). \end{cases} \tag{29}$$

To compensate for the inter-contact coupling neglected by the diagonal approximation, SCM applies a single global projection to the environment impulse at each prediction step. Using the current free velocity, define

$$\begin{aligned} &\boldsymbol{g}_{\text{env}}^{\text{step}} = \tilde{\boldsymbol{J}}_{\text{env}}\boldsymbol{v}_{\text{free}} + \tilde{\boldsymbol{\phi}}_{\text{env}}/h, \qquad \boldsymbol{\beta}_{\text{env}}^{(0)} = \max\left(-\boldsymbol{D}^{-1}\boldsymbol{g}_{\text{env}}^{\text{step}}, \mathbf{0}\right), \\ &\alpha = \text{clip}\left(-\frac{(\boldsymbol{g}_{\text{env}}^{\text{step}})^{\top}\boldsymbol{\beta}_{\text{env}}^{(0)}}{(\boldsymbol{\beta}_{\text{env}}^{(0)})^{\top}\boldsymbol{W}_{\text{env}}\boldsymbol{\beta}_{\text{env}}^{(0)}}, 0, 1\right), \\ &\hat{\boldsymbol{\beta}}_{\text{env}} = \alpha\boldsymbol{\beta}_{\text{env}}^{(0)}, \qquad \boldsymbol{s} = \alpha\boldsymbol{g}_{\text{env}}^{\text{step}}, \qquad \hat{\boldsymbol{\beta}}_{\text{env}} = \max\left(-\boldsymbol{D}^{-1}\boldsymbol{s}, \mathbf{0}\right). \end{aligned} \tag{30}$$

The coefficient $\alpha$ is shared by all environment contacts and is set to one when $\boldsymbol{\beta}_{\text{env}}^{(0)} = \mathbf{0}$. The environment impulse in (29) is replaced by $\hat{\boldsymbol{\beta}}_{\text{env}}$, with the factors $1/h$ converting impulse to force units. We use $r = 10^{-8}$ and evaluate the denominator as $\boldsymbol{z}^{\top}\boldsymbol{M}_o^{-1}\boldsymbol{z} + r\|\boldsymbol{\beta}_{\text{env}}^{(0)}\|_2^2$, where $\boldsymbol{z} = \tilde{\boldsymbol{J}}_{\text{env}}^{\top}\boldsymbol{\beta}_{\text{env}}^{(0)}$, without forming or inverting the full Delassus matrix. The projection is evaluated once per prediction step and does not introduce an additional inner iteration. Away from activation, clipping, and zero-impulse boundaries, the resulting map is piecewise rational.

This Surrogate Contact Model converts the original contact dynamics into a computationally efficient formulation, enabling (15) to be solved efficiently using gradient-based solvers.

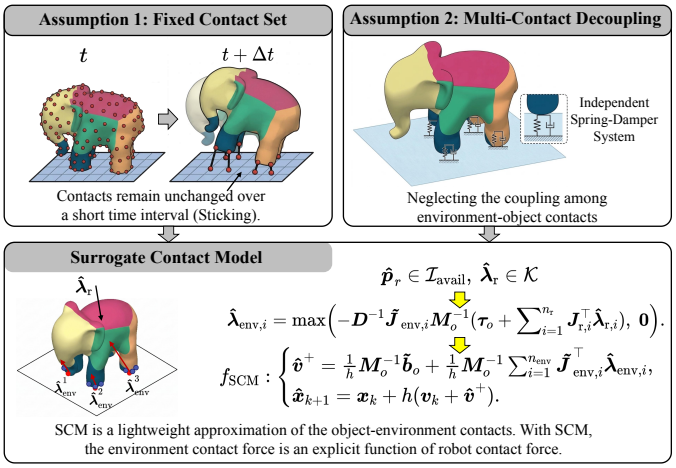



Fig. 4. Illustration of SCM. SCM is a lightweight approximation of the object-environment contact, enabling fast contact selection optimization.

### D. Formulation of CSO

Integrating SCM into (15), we can obtain the formulation of CSO as:

$$\begin{aligned} \min_{\hat{\boldsymbol{p}}_{\text{r}} \in \mathcal{I}_{\text{avail}}} \min_{\hat{\boldsymbol{\lambda}}_{\text{r}} \in \mathcal{K}} \quad & \ell_{\text{cso}}(\boldsymbol{x}_T) \\ \text{s.t.} \quad & \hat{\boldsymbol{\beta}}_{\text{env},i} = \max\left(-\boldsymbol{D}^{-1}\boldsymbol{s}_i, \mathbf{0}\right), \\ & \hat{\boldsymbol{x}}_{k+1} = f_{\text{SCM}}(\boldsymbol{x}_k, \hat{\boldsymbol{p}}_{\text{r}}, \hat{\boldsymbol{\lambda}}_{\text{r}}), \quad k = 0, \ldots, T-1, \\ & |\hat{\lambda}_{r,i}^{t1}| \leq \mu_i \hat{\lambda}_{r,i}^{n},\ |\hat{\lambda}_{r,i}^{t2}| \leq \mu_i \hat{\lambda}_{r,i}^{n}, \\ & 0 \leq \hat{\lambda}_{r,i}^{n} \leq \hat{\lambda}_{\max}^{n}, \quad i = 0, \ldots, n_{\text{r}}. \end{aligned} \tag{31}$$

Here $\boldsymbol{s}_i$ is the contact-$i$ block of $\boldsymbol{s}$ defined in (30). By solving this discrete-continuous optimization problem, we can obtain the optimal contact location among candidate contact set $\hat{\boldsymbol{p}}_r^{*} \in \mathcal{I}_{\text{avail}}$ as guidance for the lower-level contact planning. Next, we discuss the details of CSO.

[1]Proof see Appendix A.

*1) Objective:* Despite the objective of CSO being to minimize the object pose cost as denoted in (7), directly using $\ell_{\text{pose}}$ as the CSO objective $\ell_{\text{cso}}$ results in unstable contact strategies by blindly minimizing cost. In addition, the execution of the lower-level contact planning approach is inherently subject to delay, since the robot must physically approach and establish contact rather than instantaneously reaching the selected contact location. Therefore, the design of the CSO should explicitly consider the delay and avoid unstable behaviors that may cause frequent dynamic changes in the object pose. Therefore, we introduce the concept of an equivalence class of stable poses of an object as $\mathcal{X}_{\text{stab}} = \bigcup_{j=1}^{k} \mathcal{X}_j$, where $\mathcal{X}_j$ denotes the $j$-th class of stable pose of the object and $\boldsymbol{x}_{\text{o}} \in \mathcal{X}_j$ is a statically stable pose at which the object can remain at rest on the table, with its center of mass projected within the support polygon. We define the transfer cost from the current object pose to the goal stable pose as:

$$\ell_{\text{stab}}(\boldsymbol{x}_o) = d(\boldsymbol{x}_o, \mathcal{X}_g), \tag{32}$$

where $d(\cdot,\cdot)$ is a pose distance metric, $\mathcal{X}_j$ denotes the set of object poses associated with the $j$-th statically stable placement mode, and $\alpha > 0$ is a weighting coefficient. We reformulate the CSO objective to prioritize reaching the goal pose set $\mathcal{X}_g$ from the current pose $\boldsymbol{x}_o$, and then minimize $\ell_{\text{pose}}$ in $\mathcal{X}_g$:

$$\ell_{\text{cso}} = \begin{cases} \ell_{\text{stab}}, & \text{if } \boldsymbol{x}_o \notin \mathcal{X}_g, \\ \ell_{\text{pose}}, & \text{otherwise.} \end{cases} \tag{33}$$

The design of (33) effectively guides the gradient descent of CSO to select a more statically stable trajectory. Although this increases the control cost, it effectively prevents CSO from generating numerous unstable contact locations as priors, which would otherwise make the lower-level contact planning difficult to execute in practice.

*2) Solving CSO over Sampled Candidates:* The original contact dynamics in (15) are non-differentiable because of the complementarity constraints and the non-smooth temporal evolution of the contact set, rendering the constraints non-differentiable. With the SCM, (31) becomes a finite candidate search with a continuous force-optimization subproblem for each candidate. The SCM gradients on each fixed active set are

$$\frac{\partial \boldsymbol{x}_{k+1}}{\partial \boldsymbol{\lambda}_{\text{r},i}} = h\boldsymbol{M}_o^{-1}\left(h\boldsymbol{J}_{\text{r},i}^{\top} + \sum_j \tilde{\boldsymbol{J}}_{\text{env},j}^{\top} \frac{\partial \hat{\boldsymbol{\beta}}_{\text{env},j}}{\partial \boldsymbol{\lambda}_{\text{r},i}}\right). \tag{34}$$

The impulse derivative follows from (28) with fixed contact geometry, active set, and clipping branches. The resulting map is piecewise smooth; the continuous force subproblem is solved for each candidate using SNOPT, and the candidate costs are enumerated.

## IV. Contact Planning Optimization

In this section, we introduce the CPO module for contact planning. CPO accepts the optimal contact location predicted by CSO as reference, and generates approaching and contact trajectories online. We will discuss the problem of designing a prior-guided contact planning. Then we design a ranking strategy as an expressive connection between contact planning and contact prior from contact selection module. Finally, we propose a novel objective formulation for prior-guided contact trajectory generation, and present two solvers for CPO.

### A. Problem Statement

Compared to CIMPC methods, CPO must additionally consider the guidance from the high-level module and generate the corresponding executable trajectory. Based on (6), we define CPO as the single shooting problem:

$$\begin{aligned} \min_{\boldsymbol{u}_{0:N-1}} \quad & \textstyle\sum_{i=0}^{N-1} \ell_{\text{cpo}}(\boldsymbol{x}_k, \boldsymbol{u}_k, \hat{\boldsymbol{p}}_{\text{r}}^*) + V_f(\boldsymbol{x}_N) \\ \text{s.t.} \quad & \boldsymbol{x}_{k+1} = f(\boldsymbol{x}_k, \boldsymbol{u}_k), \quad k = 0, \dots, T-1, \\ & \boldsymbol{x}_k \in \mathcal{X}, \quad k = 0, \dots, T, \\ & \boldsymbol{u}_k \in \mathcal{U}, \quad k = 0, \dots, T-1. \end{aligned} \tag{35}$$

Here $\hat{\boldsymbol{p}}_{\text{r}}^*$ is explicitly included in the objective function. We next investigate two key questions: in what form should contact prior guide contact planning, and how should a feasible approach and contact trajectory be defined mathematically?

*1) Role of Contact Guidance:* In the previous section, we discussed how CIMPC methods suffer from gradient sparsity and proposed CSO, which serves as a pointwise global search on the object and provide contact guidance. But this raises another open question: in what form should the guidance affect contact execution? The most straightforward approach is full coupling, where the contact planning strictly follows and executes the guidance from the CSO. However, such a fully coupled scheme may compromise control robustness because the upper-level surrogate abstracts away detailed contact interactions. The object-centric search performed by CSO does not explicitly account for robot morphology, resulting in an inherent gap between contact selection and execution. Moreover, contact-rich manipulation is inherently underdetermined, as numerous contact-wrench and contact-location trajectories can accomplish the same task. Strictly executing the CSO-optimal trajectory would therefore restrict this intrinsic solution diversity in robot–object interaction. To address this limitation, we introduce an objective-based contact planning module named CPO. Through the ranking strategy, the CPO balances the CSO-optimal contact against nearby and other candidate contacts, establishing a loose coupling between high-level contact selection and low-level contact planning. This design benefits from global-search guidance to escape local optima while preserving the flexibility of contact planning for robust execution.

*2) Contact Switching Trajectory:* Existing CIMPC methods mainly induce nearby contacts through attractive costs, without explicitly modeling the detouring motions required for either approaching designated contacts or switching between contacts. To incorporate the upper-level guidance, we define the approaching trajectory in the contact-free space $\mathcal{S}_{\text{free}}$ as:

- All intermediate waypoints lie in the collision-free space.
- The terminal waypoint establishes contact, and the contact force lies within the friction cone.

As shown in Fig. 5, the robot–object interaction states form a low-dimensional manifold embedded in the full system state

space. A detouring motion can be viewed as planning a kinodynamically feasible bridging manifold $\mathcal{S}_{\text{approach}}$ within the contact-free space $\mathcal{S}_{\text{free}}$, connecting two states on the contact manifold $\mathcal{S}_{\text{contact}}$. This is challenging for CIMPC methods because, as discussed above, $\mathcal{S}_{\text{free}}$ provides little informative contact-induced gradient for such transitions. To naturally bridge the approaching, contact, and contact-switching phases without relying on a discrete state machine, we formulate CPO under a unified objective that jointly governs contact execution and pre-contact detouring and approach motions.

### B. Ranking Strategy of Contact Locations

In addition to the optimal contact location $\hat{\boldsymbol{p}}_{\text{r}}^{*}$ from the CSO prediction, we always consider the nearest point $\boldsymbol{p}_{\text{near}}$ to the current robot configuration as a candidate contact location:

$$\boldsymbol{p}_{\text{near}} = \arg \min_{\boldsymbol{p}_i \in \mathcal{I}_{\text{avail}}} \left\| \boldsymbol{p}_{\text{ee}} - \boldsymbol{p}_i^{\text{ee}} \right\|_2, \tag{36}$$

where $\boldsymbol{p}_i^{\text{ee}}$ is the end-effector position corresponding to contact at $\boldsymbol{p}_i$. We reformulate the problem (35) as tracking the reference contact location $\boldsymbol{p}_{\text{ref}}$, where $\boldsymbol{p}_{\text{ref}}$ is generated in real time through a ranking strategy between $\boldsymbol{p}_{\text{near}}$ and $\hat{\boldsymbol{p}}_{\text{r}}^{*}$, i.e., $\boldsymbol{p}_{\text{ref}} \in \{\boldsymbol{p}_{\text{near}}, \hat{\boldsymbol{p}}_{\text{r}}^{*}\}$. Accordingly, the ranking output determines whether to maintain the current contact or detach and switch to $\hat{\boldsymbol{p}}_{\text{r}}^{*}$ in the contact phase, and whether to establish a nearby contact or detour toward $\hat{\boldsymbol{p}}_{\text{r}}^{*}$ in the pre-contact phase. We next define the evaluation criteria for the ranking strategy.

For each candidate contact location $\boldsymbol{p}_i$, the optimal cost reduction obtained from (33) is denoted as $\Delta\ell_t^i = \ell(\boldsymbol{x}_t) - \ell(\hat{\boldsymbol{x}}_t^{+})$. Based on the predicted cost reduction range of all candidate contact locations, $\left[\Delta\ell_t^{\min}, \Delta\ell_t^{\max}\right]$, the relative ranking score of the nearby contact $\boldsymbol{p}_{\text{near}}$ is defined as

$$\rho_t = \Pi_{[0,1]}\left( \frac{\Delta\ell_t^{\text{near}} - \Delta\ell_t^{\min}}{\max(\Delta\ell_t^{\max} - \Delta\ell_t^{\min}, \epsilon)} \right), \tag{37}$$

where the fraction performs min-max normalization and $\Pi_{[0,1]}$ projects the resulting score onto $[0,1]$. The ranking score $\rho_t$ indicates the relative improvement of the nearby contact compared with all candidate contacts.

Next, we define a binary trigger variable $\kappa_t \in \{0,1\}$ to determine whether the nearby contact is suitable for replacing the CSO-selected contact:

$$\kappa_t = \mathbf{1}(\rho_t \geq \bar{\rho}), \tag{38}$$

where $\bar{\rho}$ is a predefined ranking threshold and $\mathbf{1}(\cdot)$ denotes the indicator function. When $\kappa_t = 1$, the nearby contact $\boldsymbol{p}_{\text{near}}$ provides a sufficiently competitive cost reduction and can serve as a feasible reference contact. Otherwise, the reference remains the CSO-selected contact $\hat{\boldsymbol{p}}_{\text{r}}^{*}$. The trigger variable $\kappa_t$ therefore evaluates the feasibility of the nearby contact and determines whether contact relocation is allowed.

To avoid frequent switching due to transient ranking variations, we apply a temporal hysteresis strategy to filter $\kappa_t$ when updating the reference contact.

$$\gamma_{t+1} = \begin{cases} 0, & \kappa_t = 0, \quad t_{(\kappa=0)} \geq T_1, \\ 1, & \kappa_t = 1, \quad t_{(\kappa=1)} \geq T_2, \quad t_{\text{contact}} \geq T_3, \\ \gamma_t, & \text{otherwise}, \end{cases} \tag{39}$$

**Algorithm 2** Ranking Strategy of CPO

**Input:** $\hat{\boldsymbol{p}}_r^{*}$ predicted by CSO, current robot configuration $\boldsymbol{q}_k$, threshold $\delta$
**Output:** Selected contact location $\boldsymbol{p}_{\text{ref}}$ and $\ell_{\text{lp}}$

```
Compute nearest point p_near to end-effector via (36)
Evaluate contact cost ℓ_near via (31)
Compute γ via (37), (38), (39)                ▷ Ranking strategy
if γ = 0 then
    p_ref = p̂_r^*, ℓ_lp = ℓ_lift              ▷ Lifting phase
else
    p_ref ← p_near, ℓ_lp = ℓ_place             ▷ Placing phase
end if
```

where $t_{(\kappa=0)}$ and $t_{(\kappa=1)}$ denote the consecutive time counters for the two trigger conditions, and $t_{\text{contact}}$ represents the duration of the current contact state. The filtered trigger $\gamma_t$ is subsequently used in the CPO objective formulation to determine whether the nearby contact or the CSO-selected contact should be prioritized.

The overall ranking process repeatedly evaluates the nearby contact under the current configuration. When $\gamma_t = 1$, the nearby contact $\boldsymbol{p}_{\text{near}}$ is selected as the reference contact $\boldsymbol{p}_{\text{ref}}$; otherwise, the robot continues moving toward the guidance $\hat{\boldsymbol{p}}_{\text{r}}^{*}$ generated by CSO. The complete procedure is summarized in Algorithm 2. This hierarchical design establishes a weak coupling between CSO and CPO, allowing CPO to exploit the global search guidance from CSO while retaining the flexibility to autonomously determine feasible contact strategies.

### C. Formulation of CPO

Contact planning inherently consists of multiple phases, including approaching, contact establishment, and contact switching. However, the CIMPC formulation does not naturally support contact switching, because contact switching requires leaving $\mathcal{S}_{\text{contact}}$ and traversing the contact-free space $\mathcal{S}_{\text{free}}$, where the CIMPC objective provides no effective gradient information for trajectory optimization. To address this issue, we introduce a novel objective formulation that enables multi phases planning within a unified framework. The proposed formulation is general to different task settings and compatible with various optimization solvers.

As illustrated in Fig. 6, the trajectory optimization is decomposed into three stages. First, the end-effector leaves the current contact and moves around the object toward an intermediate waypoint $\boldsymbol{p}_{\text{lift}}$ or pose $\boldsymbol{q}_{\text{lift}}$. This intermediate target guides the robot to reach a collision-free pre-contact configuration. Second, the end-effector is guided toward $\boldsymbol{p}_{\text{ref}}$ to establish contact. This design is inspired by footstep planning in legged robots [39]. The corresponding cost formulations are introduced below.

*1) Lifting Phase:* During the lifting phase, when the current nearest contact point $\boldsymbol{p}_{\text{near}}$ is infeasible, a hybrid potential field guides the robot to detach from the object and move above the

guidance. It consists of a repulsive term from the object:

$$\ell_{\text{obs}} = \begin{cases} -\log\left(\|\boldsymbol{p}_{\text{ee}} - \boldsymbol{p}_{\text{o}}\|^2 + \varepsilon\right), & \text{if } \|\boldsymbol{p}_{\text{ee}} - \boldsymbol{p}_{\text{o}}\|^2 < \sigma, \\ 0, & \text{otherwise.} \end{cases} \tag{40}$$

and an attractive term toward the lifting point $\boldsymbol{p}_{\text{lift}}$ or lifting configuration $\boldsymbol{q}_{\text{lift}}$. The lifting target satisfies: $\|\boldsymbol{p}_{\text{lift}} - \hat{\boldsymbol{p}}_r^*\|_2 = d$ or $\|\text{FK}(\boldsymbol{q}_{\text{lift}}) - \hat{\boldsymbol{p}}_r^*\|_2 = d$, which guides the robot toward a position above $\hat{\boldsymbol{p}}_r^*$. The attractive potential is defined as:

$$\ell_{\text{att}} = \|\boldsymbol{p}_{\text{ee}} - \boldsymbol{p}_{\text{lift}}\|^2 \quad \text{or} \quad \ell_{\text{att}} = \|\boldsymbol{q}_{\text{ee}} - \boldsymbol{q}_{\text{lift}}\|^2. \tag{41}$$

The lifting cost is formulated as:

$$\ell_{\text{lift}} = w_{\text{att}}\,\ell_{\text{att}} + w_{\text{obs}}\,\ell_{\text{obs}}. \tag{42}$$

This potential formulation explicitly encodes the lifting behavior and avoids optimization failure caused by gradient-sparse regions during contact breaking.

*2) Placing Phase:* During the placing phase, when the robot reaches a position above $\hat{\boldsymbol{p}}_r^*$, or when $\boldsymbol{p}_{\text{near}}$ is verified as a feasible contact location, the end-effector is guided to establish contact with the object. The placing phase is formulated by:

$$\ell_{\text{place}} = w_{\text{con}}\ell_{\text{con}} + w_{\text{o}}\ell_{\text{o}}, \tag{43}$$

where $\ell_{\text{con}}$ attracts the end-effector toward the reference contact point $\boldsymbol{p}_{\text{ref}}$, and $\ell_{\text{o}}$ attracts it toward the object center:

$$\ell_{\text{con}} = \|\boldsymbol{p}_{\text{ee}} - \boldsymbol{p}_{\text{ref}}\|^2, \quad \ell_{\text{o}} = \|\boldsymbol{p}_{\text{o}} - \boldsymbol{p}_{\text{ee}}\|^2. \tag{44}$$

Using only $\ell_{\text{con}}$ may fail to establish physical contact when the object mesh is inaccurate, since perception or projection errors can cause $\hat{\boldsymbol{p}}_r^*$ to deviate from the actual object surface. The additional term $\ell_{\text{o}}$ improves robustness against such errors.

*3) Switching Parameter:* The switching between lifting and placing behaviors is determined by the trigger $\gamma$ introduced in Section IV-B. The overall lift-and-place cost is:

$$\ell_{\text{lp}} = (1-\gamma)\ell_{\text{lift}} + \gamma\ell_{\text{place}}, \tag{45}$$

where $\ell_{\text{lp}}$ is incorporated into the stage cost $\ell_{\text{cpo}}$ in (35). By defining the terminal cost $V_f$ identical to (7), the CPO formulation is obtained as:

$$\begin{aligned} \min_{\boldsymbol{u}_k} \quad & \textstyle\sum_{i=0}^{N-1} \left( w_{\text{lp}}\ell_{\text{lp}}(\boldsymbol{x}_k, \boldsymbol{q}_k, \boldsymbol{u}_k, \boldsymbol{p}_{\text{ref}}) + w_u \|\boldsymbol{u}_k\|^2 \right) + w_{\text{cso}}\ell_{\text{cso}} \\ \text{s.t.} \quad & \boldsymbol{x}_{k+1} = f_{\text{cf}}(\boldsymbol{x}_k, \boldsymbol{u}_k), \quad k = 0, \ldots, T-1, \\ & \boldsymbol{x}_k \in \mathcal{X}, \quad k = 0, \ldots, T, \\ & \boldsymbol{u}_k \in \mathcal{U}, \quad k = 0, \ldots, T-1. \end{aligned} \tag{46}$$

The formulation in (46) preserves the favorable properties of the original CIMPC objective while enabling hybrid contact planning. Therefore, CPO can be interpreted as a prior-guided contact planning framework. We state the following lemma.

### D. Solvers of CPO

Since CPO specifies the optimization formulation rather than a specific solver, various reactive motion planners can be adopted to generate contact trajectories. Below, we present two solvers employed in our experiments.

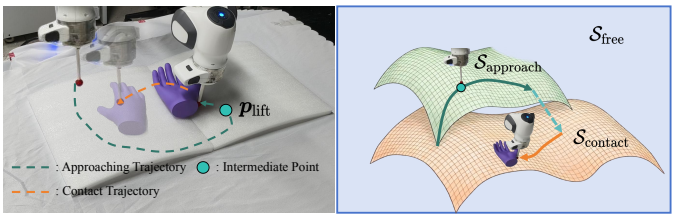



Fig. 5. (a) Illustrative example of a feasible approach and contact trajectory. (b) Explanation of CPO in a geometric perspective. CPO can be characterized as operating over two manifolds: contact states in $\mathcal{S}_{\text{contact}}$ and approaching trajectories on the manifold $\mathcal{S}_{\text{approach}}$ embedded in the contact-free state space $\mathcal{S}_{\text{free}}$. CPO must not only generate contact trajectories on $\mathcal{S}_{\text{contact}}$ but also address the gradient-sparse issue and generate trajectories on $\mathcal{S}_{\text{approach}}$.

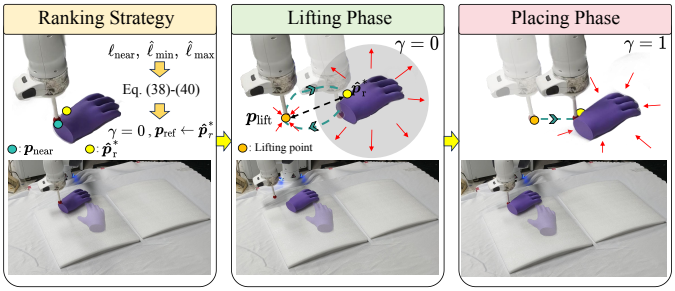



Fig. 6. Illustration of the CPO-based contact planning module. (a) Ranking strategy. When $\boldsymbol{p}_{\text{near}}$ is evaluated infeasible, $\gamma$ turns to zero and $\boldsymbol{p}_{\text{ref}} \leftarrow \hat{\boldsymbol{p}}_r^*$. The evaluation runs continuously in real-time. (b) When $\gamma = 0$, the robot is guided by the potential field to $\boldsymbol{p}_{\text{lift}}$. (c) Once above $\hat{\boldsymbol{p}}_r^*$ or the nearest point $\boldsymbol{p}_{\text{ref}}$ to the robot is detected feasible via (31), $\gamma$ becomes 1 and an attractive potential is applied, guiding the robot to contact with the object.

*1) Operational-Space Control:* We simplify the full-order contact planning problem to an end-effector point contact planning problem, avoiding high-DoF robot trajectory optimization. The resulting joint torques are generated by operational space control (OSC) [40] to track the CPO output. Specifically, the state in (46) is reduced to $\boldsymbol{s} \in \mathbb{R}^{10}$, consisting of the object position, object orientation represented by a quaternion, and the end-effector position $\boldsymbol{p}_{\text{ee}} \in \mathbb{R}^3$. Solving (46) yields the control input $\boldsymbol{u} = \Delta\boldsymbol{p}_{\text{ee}}$.

Given the desired pose $(\boldsymbol{p}_d, \boldsymbol{R}_d)$ with $\boldsymbol{p}_d = \boldsymbol{p}_{\text{ee}} + \boldsymbol{u}$ and a preset $\boldsymbol{R}_d$, the position and orientation errors are computed as $\boldsymbol{e}_{\text{pos}} = \boldsymbol{u}$, and $\boldsymbol{e}_{\text{R}} = \boldsymbol{R}_{\text{ee}}^\top \boldsymbol{R}_d$. Converting $\boldsymbol{e}_{\text{R}}$ to quaternion form gives $\boldsymbol{e}_{\text{quat}} = \left[\boldsymbol{\epsilon}_e, \eta_e\right]^\top \in \mathbb{R}^4$, and after sign correction,

$$\boldsymbol{e}_{\text{quat}} \leftarrow \begin{cases} \boldsymbol{e}_{\text{quat}}, & \eta_e \geq 0, \\ -\boldsymbol{e}_{\text{quat}}, & \eta_e < 0, \end{cases} \qquad \boldsymbol{e}_{\text{rot}} = -\boldsymbol{R}\boldsymbol{\epsilon}_e. \tag{47}$$

By defining $\boldsymbol{e}_{\text{pose}} = \left[\boldsymbol{e}_{\text{pos}}, \boldsymbol{e}_{\text{rot}}\right]^\top$, the task-space and null-space torques are given by:

$$\boldsymbol{\tau}_{\text{task}} = \boldsymbol{J}^\top(\boldsymbol{q})\left(-\boldsymbol{K}_x\boldsymbol{e}_{\text{pose}} - \boldsymbol{D}_x\dot{\boldsymbol{x}}\right), \tag{48}$$

$$\boldsymbol{\tau}_{\text{null}} = \left(\boldsymbol{I} - \boldsymbol{J}^\top(\boldsymbol{J}^\top)^\dagger\right)\left(\boldsymbol{K}_q(\boldsymbol{q}_{\text{init}} - \boldsymbol{q}) - 2\sqrt{\boldsymbol{K}_q}\,\dot{\boldsymbol{q}}\right), \tag{49}$$

and the final control torque is $\boldsymbol{\tau} = \boldsymbol{\tau}_{\text{task}} + \boldsymbol{\tau}_{\text{null}} + g(\boldsymbol{q})$. This controller is used in all simulations and real-world experiments in Section V, and is particularly suitable for tasks such as tabletop manipulation, where high orientation diversity is not critical.

*2) Sampling-based Motion Planner:* Another formulation is to directly solve the full-order planning problem of the robot. In this case, the state in (46) is given by $\boldsymbol{x} \in \mathbb{R}^{7+n_{\text{DoF}}}$, where $n_{\text{DoF}}$ denotes the number of robot DoFs. For this class of problems, we employ sampling-based MPC methods as the solver, such as MPPI [41]. Recent studies have demonstrated the impressive performance of MPPI in solving high-dimensional optimization problems for redundant robots, enabling real-time computation while avoiding issues related to gradient-based methods. The cost function is defined in the same manner as in the previous formulation, except that $\boldsymbol{p}_{\text{ee}}$ is computed via forward kinematics for the full-order problem.

## V. Experiment I: Contact Model Evaluation

To validate the predictive fidelity of the contact models, we compare MuJoCo's native quadratic-programming (QP) model, ComFree, SCM, and ContactSDF against a reference combining MJWarp collision detection and integration with fully coupled friction-pyramid LCP contact dynamics. The reference uses two $h = 0.001\,\text{s}$ substeps, while the predictors use one $h = 0.002\,\text{s}$ step. We report the linear- and angular-velocity errors of the post-step object velocity $\boldsymbol{v}^+$ relative to LCP under four contact states: contact transition, sliding, sticking, and the reference-path rollout. The benchmark uses seven objects, 32 initial states per object and contact state, and 10 rollout windows per rollout state. The results are shown in Table II.

For SCM, we smooth the initial impulse using $\text{smax}_T(x, 0) = T \log(1 + \exp(x/T))$, with $T = 10^{-4}\,\text{N s}$. It retains the same free velocity and global amplitude correction, including the clipping of $\alpha$ to $[0, 1]$, and introduces no spring-damper response time. Initial poses are obtained by native MuJoCo settling; the reported errors therefore include recovery from residual penetration in those poses.

We further apply the optimal impulse at the selected contact to control the object and evaluate whether each contact model can be used for CSO. The results are shown in Table III. Since this experiment focuses on contact-guidance feasibility, we report only success rates under synchronous inference without observation latency. All methods achieve high success rates. Although models such as MuJoCo may require fewer execution steps because of their more accurate contact modeling, SCM has a shorter inference time, which is important for obtaining timely and accurate contact information. Overall, SCM is a feasible contact model for CSO.

## VI. Experiment II: Nonprehensile Manipulation

This section evaluates the SCSP framework through nonprehensile manipulation experiments, which represent a class of contact-rich manipulation tasks characterized by frequent contact-state transitions and unstable control. The tasks consist of on-ground rotation and on-ground flipping. On-ground rotation requires the robot to move an object to a target pose on the ground through 2D pushing and rotating, whereas on-ground flipping involves 6D manipulation. We select a set of irregularly shaped objects from the ContactDB dataset [42] for validation. The objects are initialized at random poses with $x_{\text{init}}, y_{\text{init}} \in [-0.3, 0.3]$ m, and Euler angles $(\phi_{\text{init}}, \theta_{\text{init}}, \psi_{\text{init}})$ uniformly sampled from $[-\pi, \pi]$ rad. We define the conditions of task success as:

$$\|\boldsymbol{p}_o - \boldsymbol{p}_{\text{goal}}\| \le 0.02\,[\text{m}],\ 1 - (\boldsymbol{\psi}_{\text{goal}}^\top \boldsymbol{\psi}_o)^2 \le 0.015, \quad (50)$$

where the orientation tolerance corresponding to the quaternion error threshold is approximately $14.1°$. A trial is considered successful if the conditions are satisfied within 2500 rollout steps.

Simulations are conducted in `MuJoCo` [43] and `IsaacGym` [44] with a fixed integration time step $h = 0.02$s. For the sampling method in our CSO module, we use `CoACD` [45] for convex decomposition of the objects and `trimesh` for mesh processing and local frame construction. CSO is solved using batched Levenberg–Marquardt (LM), while CPO is solved using `acados` SQP-RTI with HPIPM and a solver tolerance of $10^{-4}$. All experiments are conducted on a desktop computer with an Intel i7-13620H processor, 32 GB RAM, and an NVIDIA GeForce RTX 5090 GPU.

We applied certain randomization to the environment and object parameters. The friction coefficients of the object $\mu_{\text{o}}$, fingertip $\mu_{\text{r}}$, and ground $\mu_{\text{env}}$ are all set to 0.5 with random perturbations of $\pm 0.2$. The object mass is set to 0.1 kg and randomly sampled from the discrete set $\{0.01, 0.1, 1.0, 10.0\}$ kg. We set $\boldsymbol{M}_o = \text{diag}(50, 50, 50, 0.05, 0.05, 0.05)$ and $\boldsymbol{K}_r = 300\boldsymbol{I}_7$, and the dynamic parameters used in SCM are not measured precisely, but are instead roughly set to their mean values.

For CSO, the default configuration samples $n_s = 70$ candidate contacts, retains the two lowest-cost candidates, and uses a one-step prediction with 20 batched LM iterations. CSO and CPO share a prediction step of $0.02\,\text{s}$ and pose weights $w_{\text{pos}} = 500$ and $w_{\text{quat}} = 20$. CSO uses force and tangential-force regularization weights of $0.01$ and $0.03$, respectively, with a $3\,\text{N}$ force-command limit corresponding to $\lambda^n_{\max} \approx 1.59\,\text{N}$ under the configured friction pyramid. CPO uses $N = 5$, componentwise control bounds $|u_j| \le 0.005\,\text{m}$, and weights $w_{\text{lp}} = 1$, $w_u = 50$, and $w_{\text{cso}} = 10$. Its lifting and reference-contact attraction weights are both 20, while object-center attraction and repulsive-field terms are disabled. During the flipping stage, the orientation error measures alignment of the object's target upright axis with the world vertical, leaving yaw unconstrained. Let $\boldsymbol{a}_g = \boldsymbol{R}(\boldsymbol{x}_{\text{goal}})^\top e_z$, where $e_z = [0, 0, 1]^\top$. The normalized upright error is $\ell_{\text{stab}}(\boldsymbol{x}_o) = \frac{1}{2}\left[1 - e_z^\top \boldsymbol{R}(\boldsymbol{x}_o)\boldsymbol{a}_g\right]$. The flipping-stage object cost is $\ell_{\text{cso}} = 500\ell_{\text{pos}} + 20\ell_{\text{stab}}$, with the full quaternion-error weight set to zero. Here $\ell_{\text{pos}} = \|\boldsymbol{p}_o - \boldsymbol{p}_{\text{goal}}\|^2$, except for the Stanford bunny, which uses $\ell_{\text{pos}} = \max(\|\boldsymbol{p}_{o,xy} - \boldsymbol{p}_{\text{goal},xy}\| - 0.08\,\text{m}, 0)^2$ to allow vertical motion and planar displacement within an $8\,\text{cm}$ radius during flipping.

From Table I, we select methods that are applicable to 6D manipulation and can manipulate objects with arbitrary geometries, rather than being restricted to regular shapes, as baselines, including:

- **DyWA** [17]: An end-to-end 6D nonprehensile policy evaluated on unseen object geometries using the released single-view student policy in `IsaacGym`.

TABLE II
POST-STEP VELOCITY ERRORS RELATIVE TO LCP (MEAN ± STD AND P95). LINEAR: m/s; ANGULAR: rad/s. BOLD INDICATES MINIMA.

| Contact state | MuJoCo QP | | | | ComFree | | | | ContactSDF | | | | **SCM** | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | Linear | Angular | P95 Linear | P95 Angular | Linear | Angular | P95 Linear | P95 Angular | Linear | Angular | P95 Linear | P95 Angular | Linear | Angular | P95 Linear | P95 Angular |
| Contact transition | $0.73 \pm 0.27$ | $\mathbf{4.41 \pm 4.72}$ | 1.19 | 16.33 | $0.77 \pm 0.27$ | $4.78 \pm 4.44$ | 1.19 | 15.44 | $0.36 \pm 0.16$ | $9.44 \pm 5.04$ | **0.65** | 18.27 | $\mathbf{0.23 \pm 0.18}$ | $6.31 \pm 4.38$ | 0.67 | **15.43** |
| Sliding | $0.30 \pm 0.17$ | $5.23 \pm 4.14$ | 0.69 | 15.59 | $0.34 \pm 0.17$ | $5.80 \pm 4.38$ | 0.71 | 16.66 | $\mathbf{0.12 \pm 0.10}$ | $\mathbf{3.64 \pm 2.84}$ | **0.33** | **9.16** | $0.12 \pm 0.11$ | $3.83 \pm 3.23$ | 0.40 | 10.90 |
| Sticking | $0.16 \pm 0.21$ | $1.92 \pm 2.72$ | 0.63 | 7.91 | $0.17 \pm 0.20$ | $1.92 \pm 2.71$ | 0.64 | 7.92 | $\mathbf{0.09 \pm 0.11}$ | $1.28 \pm 1.75$ | **0.36** | 5.53 | $0.11 \pm 0.14$ | $\mathbf{1.23 \pm 1.40}$ | 0.44 | **4.05** |
| Reference-path rollout | $0.03 \pm 0.10$ | $0.47 \pm 1.55$ | 0.21 | 3.22 | $0.03 \pm 0.11$ | $0.47 \pm 1.53$ | 0.24 | 3.18 | $\mathbf{0.02 \pm 0.05}$ | $0.34 \pm 1.13$ | **0.09** | 2.22 | $0.02 \pm 0.07$ | $\mathbf{0.25 \pm 0.77}$ | 0.11 | **1.73** |

TABLE III
CSO CONTACT-GUIDANCE PERFORMANCE.

| Metric | Model | | | Solver (SCM) | | | **Ours (SCM+LM)** |
|---|---|---|---|---|---|---|---|
| | M1 | M2 | M3 | CMA-ES | CEM | HPIPM | |
| Success: sync. (%) | 100 | 100 | 100 | 100 | 100 | 100 | 100 |
| Steps (sync.) | $1127.86 \pm 1312.60$ | $423.43 \pm 235.39$ | – | $640.07 \pm 382.40$ | $662.60 \pm 432.40$ | – | $318.00 \pm 245.05$ |
| Plan. (ms, sync.) | $120.77 \pm 30.38$ | $107.73 \pm 17.71$ | – | $813.01 \pm 313.17$ | $468.99 \pm 163.50$ | – | $2.55 \pm 0.22$ |

Steps and planning times: mean ± std over successful synchronous trials. Ours uses batched LM.

- **ContactSDF** [24]: A differentiable multi-contact model based on signed distance functions (SDFs), which provides closed-form collision detection and time-stepping prediction for contact-rich manipulation.
- **STOCS-3D** [23]: An offline method that jointly optimizes contact selection and a contact-implicit trajectory optimization (CITO) problem for 3D manipulation. We use STOCS-3D to generate the CITO trajectory offline and execute the resulting trajectory with operational-space control (OSC).
- **Sampling** [25]: A hierarchical method that samples candidate contacts and evaluates them with a lower-level CIMPC controller. We replace its original C3 model with $f_{\text{cf}}$ for a common contact model.
- **CF-MPC** [22]: A complementarity-free MPC method with a closed-form contact model and limited explicit contact-selection capability.

### A. Robot Manipulation in Simulation

We further evaluate SCSP in redundant robot manipulation tasks. Compared with a floating end-effector, contact problems for redundant robots introduce complex kinematic constraints. In the experiments, we demonstrate the superiority of SCSP and validate the soundness of our framework.

Due to the strong dependencies of the source code released for DyWA, which are difficult to transfer, we follow its setup and conduct all experiments in `IsaacGym` for consistency, with the same simulation settings described above. All methods use our link-mounted sphere as the end-effector to replicate point contacts. Since DyWA uses the Franka gripper, we adjust the link length so that the sphere's center coincides with the gripper's center. The Cartesian stiffness $\boldsymbol{K}_x = \text{diag}(1000, 1000, 1000, 50, 50, 50)$ and $\boldsymbol{D}_x = \sqrt{2\boldsymbol{K}_x}$. All methods use the same settings as above, except that $\boldsymbol{u} \in \mathbb{R}^7$ in I-MPC, CF-MPC and A-MPC. $\boldsymbol{p}_{\text{ee}}$ is obtained via differentiable forward kinematics.

The experimental results are shown in Fig. 7, Fig. 8, and Table IV. SCSP achieves a higher success rate than other model-based algorithms and outperforms DyWA on all

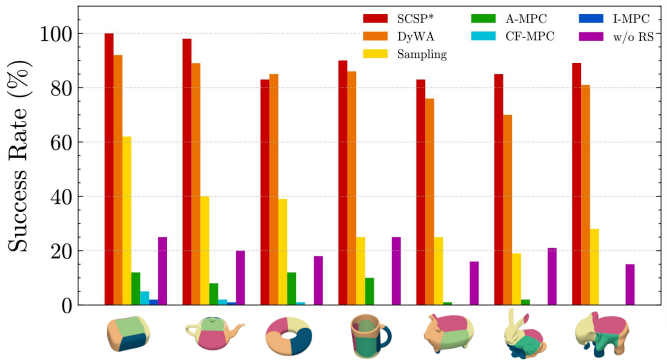



Fig. 7. Experimental results of robot manipulation in simulation. SCSP significantly outperforms other model-based methods and exceeds the performance of well-trained learning-based methods.

objects except the torus. Additionally, SCSP achieved the shortest execution time, demonstrating that SCSP can generate correct contact locations through CSO, thereby efficiently minimizing the pose cost of the object. SCSP also achieved the smallest convergence position error and quaternion error. Furthermore, we observed that during experiments, DyWA tends to flip objects by pressing the edges of the top surface, whereas our method makes more comprehensive decisions regarding whether to lift the object by pushing the bottom or by pressing the top. Although many flipping tasks can be accomplished through pressing, for certain objects (such as elongated objects), the pressing approach requires applying substantial torque to the object and demands sufficiently high coefficients of friction. Based on our observations, the global contact selection mechanism in CSO enables SCSP to exhibit greater diversity in its manipulation behaviors compared with other methods, rather than converging to a single strategy.

In addition, CF-MPC and A-MPC can be regarded as ablations of the contact selection strategy. CF-MPC performs contact planning without contact selection, whereas A-MPC replaces the physical reasoning of CSO-based contact planning with a simple prior. Compared with SCSP, both methods show a substantial decrease in success rate, indicating that reasonable contact selection is crucial for complex 6D nonprehensile manipulation tasks. The ranking strategy is evaluated separately below.

We further validated the impact of different numbers of sampled candidate points on performance. As shown in Table V, when the number of sampled points reaches 70, nearly the highest success rate is achieved with relatively low SCSP computation time. As the number of sampled points continues to increase, the success rate plateaus and computational effi-

TABLE IV
COMPARISON AND ABLATION RESULTS FOR NONPREHENSILE MANIPULATION TASKS IN THE ROBOT EXPERIMENTS (200 TRIALS PER TASK)

| Task | Metric | Comparison | | | | | | Ablation | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | SCSP* | DyWA | ContactSDF | STOCS-3D | Sampling | CF-MPC | A1 | A2 | A3 | A4 | A5 | A6 | A7 |
| Rotation | Success Rate | **0.95** | 0.86 | – | – | 0.56 | 0.01 | 0.36 | 0.36 | 0.10 | 0.38 | 0.14 | 0.19 | 0.10 |
| | Execution Time (s) | **15.16** | 18.12 | – | – | 18.97 | 21.18 | 19.36 | 19.36 | 18.32 | 35.00 | 212.93 | 2.65 | 3.91 |
| | Planning Time (s) | 0.024 | 0.031 | – | – | 0.228 | 0.082 | 0.023 | 0.023 | **0.019** | 0.169 | 0.891 | 0.042 | 0.013 |
| | Obj. Pos. Error (m) | **0.009** | 0.015 | – | – | 0.014 | 0.016 | 0.019 | 0.019 | 0.016 | 0.121 | 0.340 | 0.446 | 0.291 |
| | Obj. Quat. Error | **0.015** | 0.019 | – | – | 0.016 | 0.026 | 0.031 | 0.388 | 0.611 | 0.565 | 0.349 | | |
| Flipping | Success Rate | **0.81** | 0.69 | – | – | 0.39 | 0.01 | 0.26 | 0.00 | 0.37 | 0.19 | 0.05 | 0.40 | 0.00 |
| | Execution Time (s) | $21.79 \pm 9.79$ | 20.56 | – | – | **19.68** | 22.56 | $22.05 \pm 9.36$ | – | $23.60 \pm 11.86$ | 177.90 | 1409.24 | 36.65 | – |
| | Planning Time (s) | $0.023 \pm 0.003$ | 0.031 | – | – | 0.252 | 0.081 | $0.025 \pm 0.003$ | $0.013 \pm 0.000$ | $0.022 \pm 0.007$ | 0.135 | 0.738 | 0.043 | 0.013 |
| | Obj. Pos. Error (m) | $0.055 \pm 0.068$ | 0.016 | – | – | **0.012** | 0.018 | $0.157 \pm 0.140$ | $0.148 \pm 0.050$ | $0.122 \pm 0.124$ | 0.132 | 0.372 | 0.0193 | 0.347 |
| | Obj. Quat. Error | $0.204 \pm 0.345$ | 0.021 | – | – | 0.022 | 0.027 | $0.383 \pm 0.412$ | $0.734 \pm 0.316$ | $0.473 \pm 0.414$ | 0.399 | 0.640 | **0.0043** | 0.800 |

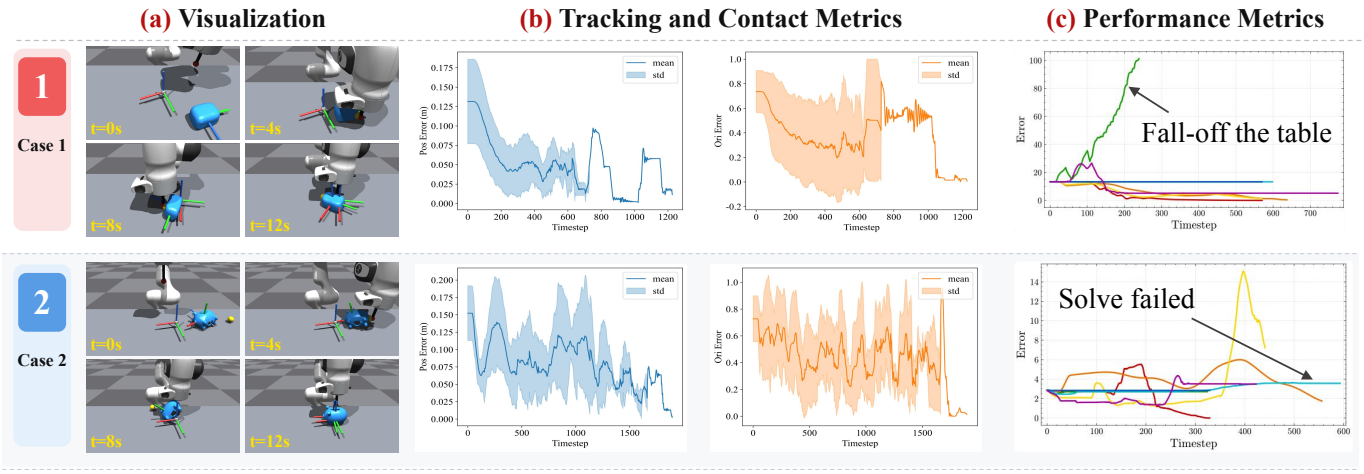



Fig. 8. Nonprehensile manipulation experiments in simulation. (a): Snapshots of SCSP in the robot experiments, where the goal pose is visualized either transparently or as a floating coordinate frame. (b): The mean and standard deviation curves of the position and orientation errors over steps across 10 SCSP trials. (c) The pose error $\ell_{\text{pose}}$ of different methods during manipulation. The comparison methods either require longer execution time or fail due to local minima or solver failures.

TABLE V
EVALUATION OF THE EFFECT OF THE NUMBER OF CANDIDATE POINTS

| Sampling numbers in CSO | 5 | 10 | 70 | 120 | 500 |
|---|---|---|---|---|---|
| **Avg. Success Rate** | 0.25 | 0.59 | **0.88** | **0.88** | 0.85 |
| **SCSP Computation Time** (s) | **0.012** | 0.017 | 0.025 | 0.032 | 0.089 |

ciency deteriorates. This indicates that increasing the number of sampled points does not necessarily improve performance. There exists an optimal convergence value. For nonprehensile manipulation tasks, 70 to 100 sampled points are sufficient to characterize the geometric features of object geometries.

### B. Ablation Studies

We ablate each component under the same experimental setup as in Section VI-B.

*1) Functional Analysis of SCSP Components:* **Ranking strategy (A1–A3):** CPO strictly follows CSO-optimal guidance $p_r^*$ (A1), always executes the nearest contact $p_{\text{near}}$ (A2), or follows simple heuristic guidance (A3).

**CSO (A4–A5):** A4 replaces soft-SCM with MuJoCo's QP-based contact model; A5 solves soft-SCM using CMA-ES. Both use asynchronous inference.

**CPO (A6–A7):** A6 removes the phase-dependent attraction and repulsion potentials and directly optimizes motion trajectories; A7 directly explores contact trajectories using MPPI [1] with the cost $\ell_{\text{A7}} = \omega_{\text{o}}\ell_{\text{o}} + \omega_{\text{cso}}\ell_{\text{cso}}$,

Table IV reports the available results. Rotation A1 uses the historical w/o RS Franka results, in which CPO strictly follows the CSO-generated contact guidance. Flipping A1–A3 use the current seven-object cohort.

For rotation, the ranking strategy mainly affects execution efficiency rather than success rate, whereas it has a significant effect on success rate for flipping. The success rates of A4 and A5 both decrease substantially. Together with the experiments in Section V, this suggests that the contact guidance remains correct, while perception latency is more likely to cause failures. A6 has only a slight effect on success rate; in the failure episodes we inspected, most failures were false contact detections without actual contact. A7 substantially increases computation time, causing observation latency that degrades success rate.

*2) Counterfactual Analysis of the Ranking Strategy:* A1–A3 evaluate alternative contact-selection policies at the task level. We further assess the ranking strategy at individual decision points using matched counterfactual rollouts.

Let $r_T = \mathbb{I}(\text{success})$ be the terminal reward. For a state $s_t$ and ranking decision $b$, the discounted rollout value is

$$J(s_t, b) = \mathbb{E}_{\tau \sim \pi(\cdot|s_t,b)} \left[\gamma^T \mathbb{I}(\text{success})\right], \qquad \gamma = 0.99, \quad (51)$$

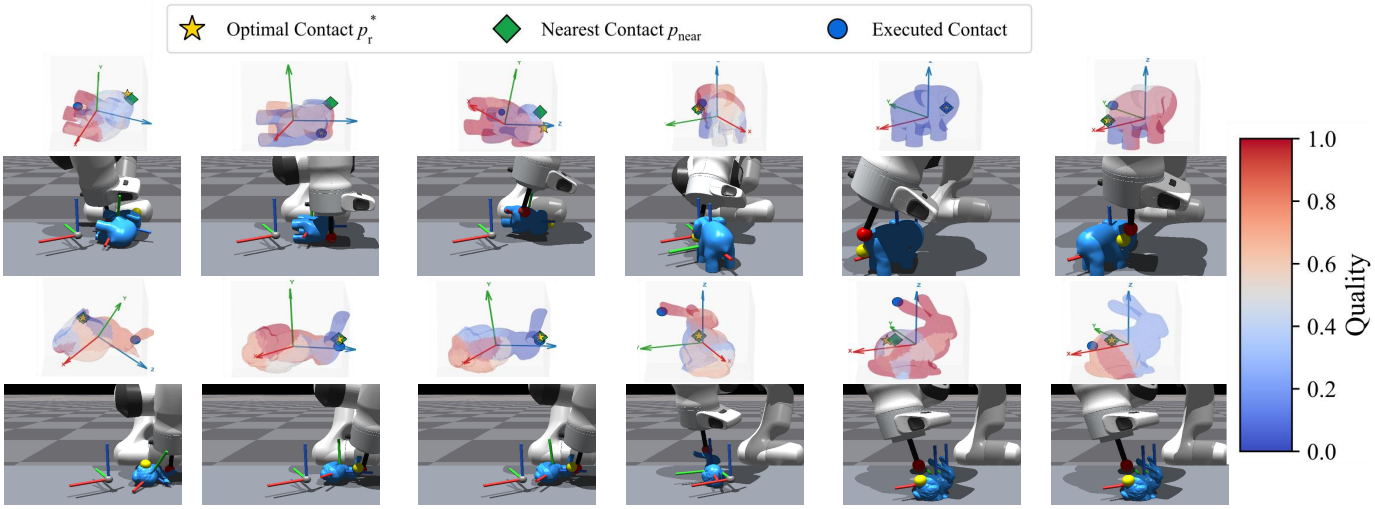



Fig. 9. Visualization of the contact map during flipping elephant and bunny task.

where $T$ is the number of remaining control steps.

For each object and task, we reconstruct one successful synchronous trajectory using seeds drawn from existing successful-rollout records, and uniformly sample 5 distinct states with measured physical contact, excluding transit and detour motion, where the CSO guidance differs from the nearby contact. The simulator and controller histories are reconstructed by replaying the complete prefix, checking observations, commands, and ranking decisions up to the sampled state. We compare the ranking-strategy decision with a counterfactual rollout that makes the opposite decision at that state, while keeping subsequent replanning and task parameters unchanged. Let $J_{\mathrm{RS}}$ and $J_{\mathrm{cf}}$ denote the two rollout values at $s_t$. Their normalized advantage is

$$\widetilde{A}(s_t) = \begin{cases} \dfrac{J_{\mathrm{RS}} - J_{\mathrm{cf}}}{\max\{J_{\mathrm{RS}}, J_{\mathrm{cf}}\}}, & \max\{J_{\mathrm{RS}}, J_{\mathrm{cf}}\} > 0, \\ 0, & \text{otherwise.} \end{cases} \tag{52}$$

Here $\widetilde{A} \in [-1, 1]$, and $\widetilde{A} > 0$ indicates that the ranking strategy is preferred. A value of 1 $(-1)$ means that only the RS (counterfactual) rollout succeeds. For two successful rollouts, this normalization depends only on the difference in their remaining completion steps, removing the common discount scale associated with long remaining trajectories.

We report contact-switching counts (CSC), counting a switch only when the robot leaves the previous contact and establishes a distinct contact patch. The probabilities $P(p_{\mathrm{near}})$ and $P(p_r^*)$ use all eligible binary contact decisions in the original successful trajectories. The nearest-contact selection frequency $f_{\mathrm{near}}$ is the number of nearest-contact selections per simulated second. The CSO-guidance switching frequency $f_r^*$ counts changes of $p_r^*$ per simulated second, using a 1 cm displacement threshold in the object's frame to exclude rigid object motion and mesh jitter. Counts and frequencies use complete original trajectories, rather than the sampled counterfactual branches.

Because SCSP replans at every control cycle, many single-decision changes do not affect the final outcome. We normalize each sampled pair before aggregation and retain all 5 sampled states per trajectory, including $\widetilde{A} = 0$, when calculating $P(\widetilde{A} > 0)$ and the mean and sample standard deviation of $\widetilde{A}$ in Table VI.

TABLE VI
RANKING-STRATEGY COUNTERFACTUAL ANALYSIS.

| Task | CSC | $P(p_{\mathrm{near}})$ | $P(p_r^*)$ | $f_{\mathrm{near}}$ (Hz) | $f_r^*$ (Hz) | $P(\widetilde{A} > 0)$ (%) | $\widetilde{A}$ |
|---|---|---|---|---|---|---|---|
| Rotation | – | – | – | – | – | – | – |
| Flipping | – | – | – | – | – | – | – |

CSC: total completed physical contact switches; $f_{\mathrm{near}}$: nearest-contact selections per second; $f_r^*$: CSO-guidance patch switches per second; $\widetilde{A}$: normalized advantage, mean $\pm$ sample standard deviation over all sampled decisions; $--$: pending measurements under the revised protocol.

We also test the sensitivity to the sliding-window parameters $T_1$, $T_2$, and $T_3$. Each parameter is independently scaled by $\{0.5, 0.75, 1.0, 1.25, 1.5\}$, with the others fixed at their default values. We report the success rate and execution time under the same experimental conditions; the results are shown in Fig. .

### C. Real-world Experiments

Finally, we validate the robustness of SCSP to in real-world experiments. Most model-based methods perform well in simulation, but in real-world experiments, they often rely on highly accurate perception systems, such as motion capture. The main challenge in replacing exact perception with purely vision-based perception is that errors in pose observation can perturb contact estimation, leading to incorrect inference of contact states and consequently inaccurate contact dynamics. To the best of our knowledge, our method is the first model-based approach that can achieve purely vision-based, complex geometries, model mismatch contact-rich manipulation.

*1) Experimental Setup:* We conduct experiments using a Franka Emika Panda equipped with a 3D-printed end-effector, and arbitrarily select 4 irregularly shaped objects: Duck, Bunny, Hand, and Xbox Controller, as shown in Fig. 10.

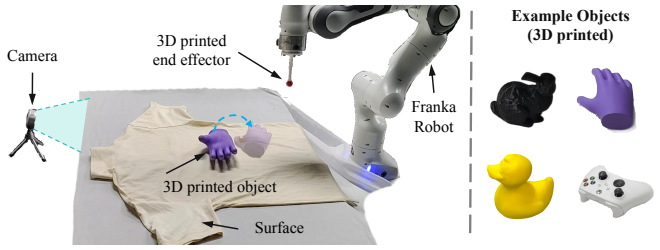



Fig. 10. Hardware setup of the real-world experiments.

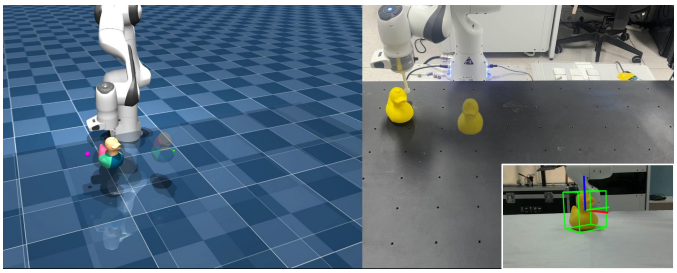


Fig. 11. Perception method in the real-world experimental setting. Object pose is detected using `FoundationPose` and filtered via a Kalman filter. Contact estimation is performed using a `MuJoCo` implementation that approximates the experimental environment.

We evaluate on-ground rotation on Duck and Bunny, and on-ground flipping on Bunny, Hand, and Xbox Controller. The object CAD models are obtained from the ContactDB dataset and online repositories. To validate the robustness of our method under inaccurate dynamics, we provide only rough estimates of the object's physical parameters such as mass and friction coefficients. Moreover, instead of relying on precise motion capture for exact perception, we perform state estimation by `FoundationPose` [46] using a single-view camera (Realsense D435i). In real-world experiments, precise object–environment contact locations are not directly observable. To address this, we approximately reconstruct the task environment in `MuJoCo` for contact estimation, as shown in Fig. 11. We perform a simple calibration procedure to determine the object's scale parameter and positional offsets in the simulator to eliminate contact estimation misclassification. Specifically, we use Franka's kinesthetic teaching mode to move the end-effector and make contact with several randomly selected locations on the object, and then tune the object's scale and the offsets $[\Delta x, \Delta y, \Delta z]$ such that the simulator can approximately predict whether the end-effector is in contact with the object. The goal pose range, success conditions, and maximum execution steps for the task are identical to those in the simulation experiments. Sampling and DyWA use the same settings as in the previous section, and the outputs of DyWA are executed using an impedance controller.

The OSC controller is implemented in `C++`. Unlike in simulation experiments, control in real-world deployment is subject to disturbances, leading to control lag and steady-state errors. To address this, we designed an adaptive controller for precise control, which we refer to as the Incremental OSC. We introduce an intermediate position vector $\boldsymbol{p}_{\text{inc}} = \boldsymbol{p}_{\text{inc}} + \boldsymbol{K}_{\text{inc}}\boldsymbol{u}$, and replace $\boldsymbol{e}_{\text{pos}}$ with $\boldsymbol{e}_{\text{pos}} = \boldsymbol{p}_{\text{inc}} - \boldsymbol{p}_{\text{ee}}$. The steady-state error is adaptively eliminated under the high-frequency self-integrating effect. In the experiment, we set $\boldsymbol{K}_{\text{inc}} = 0.01\boldsymbol{I}_3$, $\boldsymbol{K}_x = \text{diag}(500, 500, 500, 10, 10, 10)$ and $\boldsymbol{D}_x = \sqrt{2\boldsymbol{K}_x}$.

*2) Results and Discussion:* We first evaluate the success rate of SCSP across different objects and tabletop materials to assess its robustness under model inaccuracies. The experimental results are shown in Fig. 12, 13, and Table VII. They indicate that our method maintains strong performance even under perception noise and model mismatch.

We conducted each experiment on two different tabletop materials: smooth and rough as in Fig. 14. For the on-ground rotation experiments, the smooth surface denotes a tabletop with low friction, whereas the rough surface denotes cotton clothes. For the on-ground flipping experiments, which require greater friction, we use foam padding as the rough surface and cotton cloth as the smooth surface. Note that our method does not measure friction coefficients, object mass, or other physical parameters, but instead uses coarse estimates. Additionally, biases in contact estimation and noise in object pose estimation can interfere with the decision-making of SCSP. The experiment results suggest that SCSP can effectively cope with model inaccuracies and disturbances, which is typically challenging for model-based methods.

As shown in Table VII, SCSP achieved a 99.2% success rate (119/120) in the on-ground rotation experiments with complex object geometries and also showed strong performance in the flipping tasks, whereas the comparison methods failed in nearly all flipping experiments. We observe that the failures of DyWA mainly arose from contact slippage and violations of the Franka acceleration limits during pressing, while the failures of Sampling were primarily caused by the selection of unstable contact locations. Due to the complex geometry and relatively smooth material of the 3D printed objects, the fingertip sometimes fails to establish stable contact with the object, resulting in relative sliding that causes the execution of the contact plan planned by SCSP to fail. Additionally, surface friction also affects the success rate of flipping. As shown in the results, SCSP achieves a notably higher success rate on the rough surface compared with the smooth surface.

We further conduct continuous manipulation experiments, as shown in Fig. 13. During manipulation, we repeatedly perturb the object pose to evaluate the real-time performance, closed-loop planning capability, and robustness of SCSP under accumulated perception errors. Under repeated perturbations, both the pose estimated from `FoundationPose` and the contact estimation become biased. However, SCSP exhibits strong replanning capability and consistently completes the continuous manipulation tasks under disturbances.

We combine the preceding experimental results to analyze the source of SCSP's robustness to model mismatch. We conclude that: 1) the ablation study reveals that optimality of the contact prior is not necessary. Although the prior is important for contact planning, even a coarse contact location selection as a prior can still bring substantial performance improvement. This observation also provides insight into the role of the contact-selection model: how to properly connect the prior with the planning framework seems to be more important than the optimality and accuracy of the prior itself.

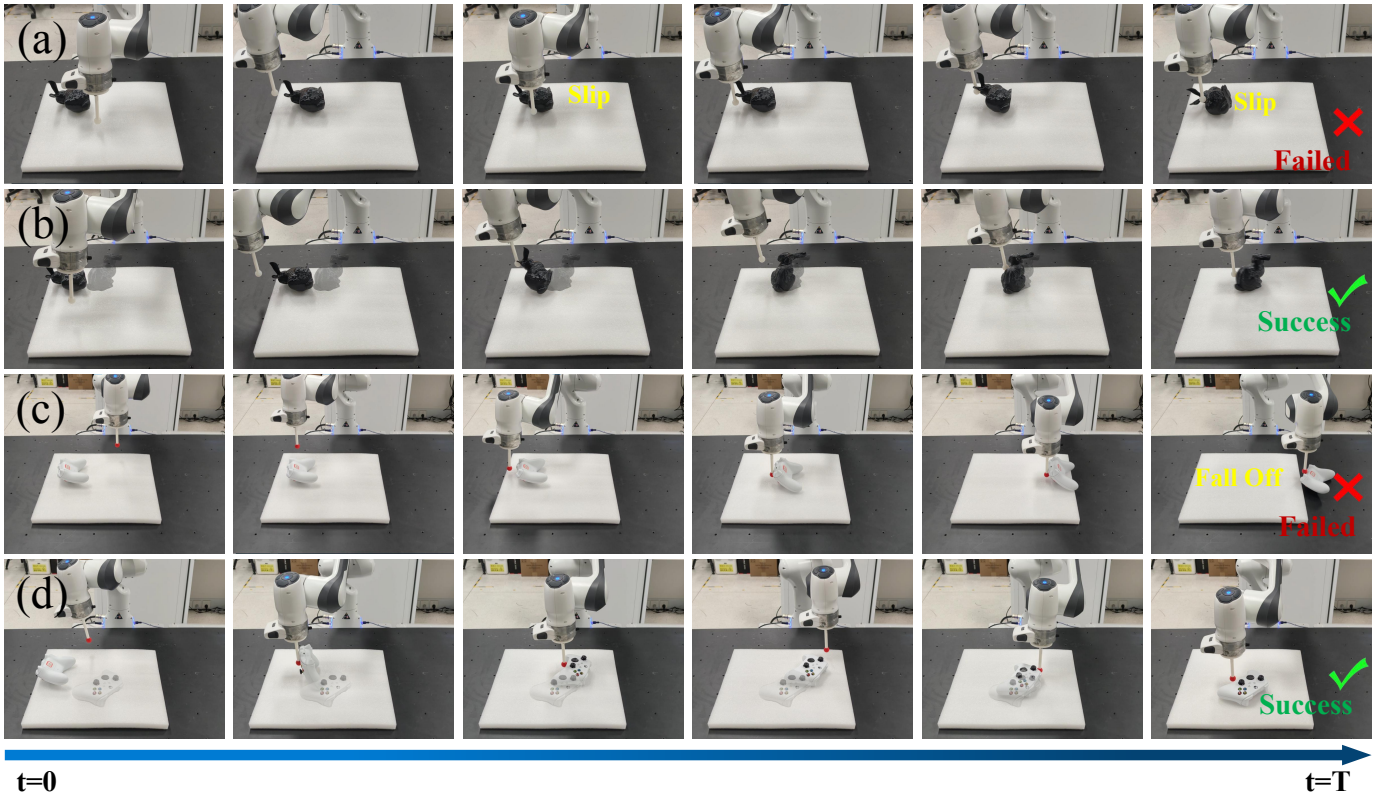



Fig. 12. Snapshots of nonprehensile manipulation in real-world experiments. (a) **DyWA** failed to flip the bunny. (b) **SCSP** succeeds in flipping the bunny. (c) **Sampling** failed by pushing the Xbox controller to fall off the foam padding. (d) **SCSP** succeed in flipping the Xbox controller.

TABLE VII
REAL-WORLD MANIPULATION PERFORMANCE ON SMOOTH AND ROUGH SURFACES (30 TRIALS PER CONDITION).

| Surface | Metric | Rotation | | | | | | Flipping | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | | Duck | | | Bunny | | | Bunny | | | Hand | | | Xbox | | |
| | | Sampling | DyWA | **SCSP*** | Sampling | DyWA | **SCSP*** | Sampling | DyWA | **SCSP*** | Sampling | DyWA | **SCSP*** | Sampling | DyWA | **SCSP*** |
| Smooth | SR (%) | 60 [42.3, 75.4] | 50 [33.2, 66.8] | **100** [88.6, 100.0] | 40 [24.6, 57.7] | 40 [24.6, 57.7] | **100** [88.6, 100.0] | 0 [0.0, 11.4] | 0 [0.0, 11.4] | **46.7** [30.2, 63.9] | 13.3 [5.3, 29.7] | 0 [0.0, 11.4] | **70** [52.1, 83.3] | 10 [3.5, 25.6] | 13.3 [5.3, 29.7] | **90** [74.4, 96.5] |
| | FR (%) | 6.7 | 10 | **0** | 6.7 | 10 | **0** | 20 | **0** | **0** | 23.3 | 6.7 | **0** | 16.7 | 6.7 | **0** |
| | TR (%) | 33.3 | 40 | **0** | 53.3 | 50 | **0** | 80 | 100 | **53.3** | 63.4 | 93.3 | **30** | 73.3 | 80 | **10** |
| | ET (s) | 15.31 ± 10.17 | **15.19 ± 10.31** | 19.25 ± 9.58 | 17.12 ± 8.83 | **14.57 ± 5.69** | 15.56 ± 7.81 | – | – | **30.34 ± 12.41** | **20.19 ± 11.59** | – | 27.99 ± 9.73 | 19.86 ± 2.15 | 21.31 ± 9.67 | **18.90 ± 13.26** |
| Rough | SR (%) | 53.3 [36.1, 69.8] | 46.7 [30.2, 63.9] | **96.7** [83.3, 99.4] | 40 [24.6, 57.7] | 36.7 [21.9, 54.5] | **100** [88.6, 100.0] | 0 [0.0, 11.4] | 0 [0.0, 11.4] | **66.7** [48.8, 80.8] | 10 [3.5, 25.6] | 10 [3.5, 25.6] | **83.3** [66.4, 92.7] | 16.7 [7.3, 33.6] | 20 [9.5, 37.3] | **90** [74.4, 96.5] |
| | FR (%) | 6.7 | 13.3 | **0** | 3.3 | 6.7 | **0** | 10 | 20 | **0** | 20 | 10 | **0** | 16.7 | 16.7 | **0** |
| | TR (%) | 40 | 40 | **3.3** | 56.7 | 56.6 | **0** | 90 | 80 | **33.3** | 70 | 80 | **16.7** | 66.6 | 63.3 | **10** |
| | ET (s) | 18.08 ± 8.01 | **16.88 ± 8.90** | 19.25 ± 9.58 | 18.41 ± 6.73 | 25.25 ± 10.05 | **15.56 ± 7.81** | – | – | **30.34 ± 12.41** | 26.61 ± 11.32 | **20.30 ± 5.83** | 27.99 ± 9.73 | **18.39 ± 11.86** | 31.08 ± 11.35 | 18.90 ± 13.26 |

SR: success rate; brackets below SR report the two-sided Wilson 95% confidence interval (%), computed separately from 30 trials for each method–task–object–surface condition; FR: fall-out rate; TR: timeout rate (shown as 100 minus the displayed SR and FR to preserve a total of 100%); ET: execution time in seconds. DyWA/Sampling ETs for Rotation and for Flipping Hand and Xbox are synthetic estimates from mixed successful SCSP simulation trajectories. Bold denotes the highest SR or the lowest FR, TR, and mean ET among methods under the same condition. −− indicates unavailable measurements.

TABLE VIII
SCSP EXECUTION EVENTS AND SUBSEQUENT TASK COMPLETION OVER 300 REAL-WORLD TRIALS.

| **Slippage** | | **Contact-prediction error** | | **Unintended toppling** | |
|---|---|---|---|---|---|
| Count | CR (%) | Avg. count | CR (%) | Count | CR (%) |
| 28 | 89.3 (25/28) | 2.82 | 84.3 (253/300) | 15 | 93.3 (14/15) |

The designs of CSO and the ranking strategy are both developed with this perspective. 2) CPO has a strong closed-loop nature. The real-time performance and robustness of CIMPC methods have been thoroughly validated in previous work. CPO is a prior-guided contact planning framework built upon the CIMPC formulation. It inherits the robustness of CIMPC while improving manipulation diversity. Therefore, the SCSP framework is able to achieve robust performance.

## VII. GENERALIZATION TO MULTIPLE TASKS

We further implement SCSP in different scenarios to evaluate its generalization. Using multi-contact tasks and long-horizon tasks as examples, we demonstrate that SCSP can perform online contact selection and execute more complex manipulation tasks on more underactuated objects.

### A. Case Study I: Multi-Contact Task

SCSP can be easily extended to multi-contact tasks. We illustrate the generalization of SCSP with following example:

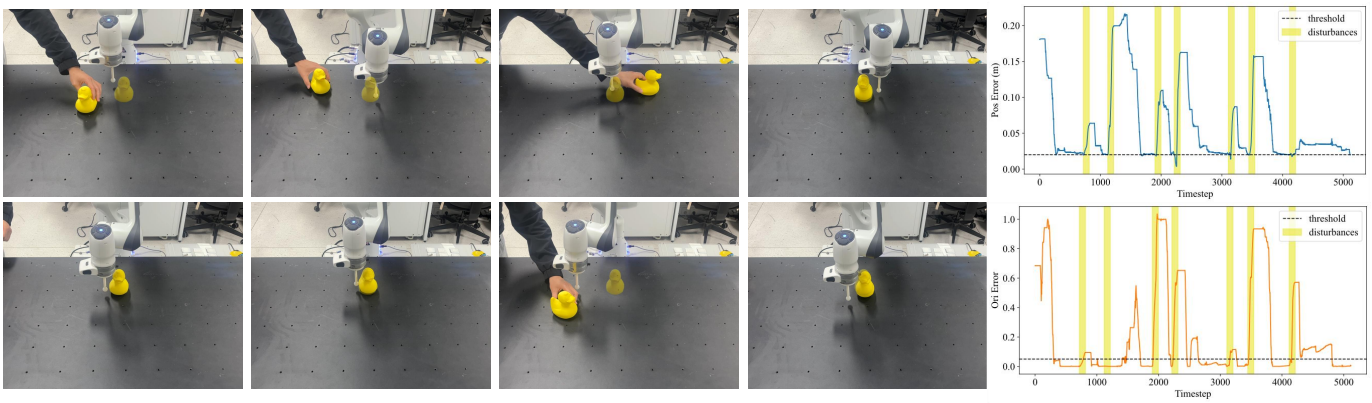



Fig. 13. Snapshots of manipulation under external disturbances. The object's pose is continuously perturbed during manipulation to demonstrate SCSP's real-time robustness under such disturbances. The panels on the right shows the temporal evolution of position and orientation errors.

TABLE IX
COMPUTATION TIME OF SCSP

| Metric | Value |
|---|---|
| **CSO Computation time** | |
| Offline sampling cost time (s) | $4.82 \pm 3.85$ |
| Exchange cost time (ms) | $0.002 \pm 0.01$ |
| MIQP cost time (ms) | $0.32 \pm 0.05$ |
| CSO solving time (ms) | $1.07 \pm 0.32$ |
| **CPO Computation time** | |
| RS cost time (s) | $0.01 \pm 0.00$ |
| Average CPO solving time (ms) | $8.68 \pm 0.31$ |




Fig. 14. Robustness validation of SCSP on surfaces with different friction. (a) Foam. (b) Cotton clothes.

*1) Soccer Dribbling:* We use a soccer dribbling task to demonstrate the performance of SCSP in dynamic scenarios. In this task, SCSP plans contact with the ball using one of the two feet and drives it toward the goal through multi-contact manipulation. The main challenges lie in the highly dynamic behavior of the ball, which makes stable and sustained contact difficult, and in coordinating the two feet effectively. We read the ball position $\boldsymbol{p}_k$ and estimate its velocity $\boldsymbol{v}_k$ from the simulation using a Kalman filter. The 3D model of the ball is then input to the SCSP to compute the actions $\boldsymbol{u} \in \mathbb{R}^{2\times 4}$ for both feet, where the control vector for each foot is $[x, y, z, \theta_{\text{yaw}}]$. We define activation triggers separately for the left and right feet: using the midpoint between both feet as the center, the space is divided into left and right half-spaces. When $\boldsymbol{x}_k$ appears in a half-space, the corresponding left or right foot is activated. Additionally,

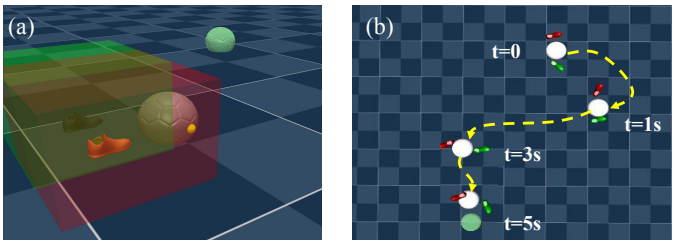



Fig. 15. Soccer dribbling task. (a) Experiment setup. The task is to kick the soccer to the target (translucent). Red and green cubes show footworkspaces, and the yellow point indicates CSO-planned contact locations. (b) Gribbling trajectory.

workspace constraints (represented as semi-transparent cubes) are imposed on each foot to prevent singular gait patterns. The number of CSO sampling points is set to $n_s = 70$, and points close to the ground $\{p_i \in \mathcal{I}_{\text{obj}} \mid z_i < 0.02\text{m}\}$ are masked out. We randomly selected 10 goal positions for the soccer ball, and SCSP continuously controlled the robot's feet to dribble the ball to each target, achieving a 100% success rate. The experimental results are shown in Fig. 15. This experiment serves as a toy example, simulating the approaching and contact phases in humanoid soccer dribbling. In the future, we plan to extend SCSP to more complex motion planners to realize a more complete demonstration [47], [48].

*2) Tilted Pushing:* SCSP can be extended to manipulation tasks in uneven environments, such as tilted pushing. Stable object poses on a slope are inherently fragile, requiring accurate contact location selection and online replanning. We compare open-loop methods, which plan a contact sequence offline, with SCSP, which replans online during execution. As the baseline, we use STOCS-3D to compute an offline sequence of contact locations and contact forces, and execute this sequence using our CPO. Experiments are conducted in `IsaacGym` with a 15° slope and a friction coefficient of 0.1. For SCSP, we set $w_{\text{stab}} = 0$ and $\hat{\lambda}^n_{\text{max}} = 0.5$, while keeping all other parameters the same as in Section VI-A. We selected the teapot and duck (used in previous experiments) and randomly set 10 stable poses on the slope as goal poses, and pushed and rotated the objects from below to reach these goals. The results in Table X and Fig. 16 show that the real-time replanning

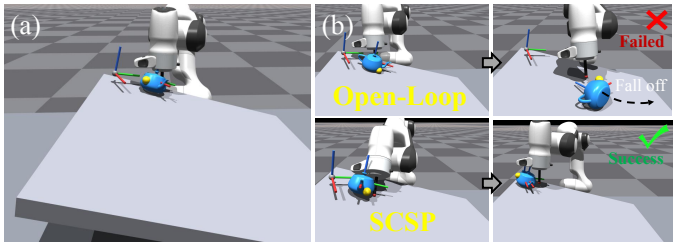



Fig. 16. Experiment setup. The task is to push an object on a surface tilted at 15°. (a) The open-loop method fails when execution errors occur, causing the teapot to fall off the tilted surface. (b) SCSP can replan online and thus successfully complete the dynamic task.

capability of SCSP provides substantially better robustness.

*3) Grasping & Dexterous Manipulation:* SCSP can also be applied to predict grasping poses, such as bimanual grasping and multi-finger grasping. Following [49], we use the Grasp Wrench Boundary (GWB) [50] to select grasping regions and identify the top-K pairs of potential valid regions for grasping. Subsequently, sampling of the candidate set $\mathcal{I}_{\text{obj}}$ is performed within these regions. The objective of the CSO inner loop optimization is then transformed to force closure:

$$\begin{aligned} \min_{\hat{\boldsymbol{\lambda}}_r \in \mathcal{K}} \quad & \sum_{j=1}^{N_d} \left\| \xi \boldsymbol{w}^{(j)} - \boldsymbol{J}_o \hat{\boldsymbol{\lambda}}_r^{(j)} \right\|_2^2 + w_\lambda \sum_{j=1}^{n_d} \left\| \hat{\boldsymbol{\lambda}}_r^{(j)} \right\|_2^2 \\ \text{s.t.} \quad & \sum_{i=1}^{K} \hat{\lambda}_r^{(j),n} \geq \zeta, \ \forall j = 1, \ldots, n_d. \end{aligned} \tag{53}$$

where $n_c$ is the number of robot contact points, $n_d$ denotes the number of sampled disturbance wrenches, $\boldsymbol{w}^{(j)} \in \mathbb{R}^6$ represents the $j$-th disturbance wrench. $\boldsymbol{\lambda}_r^{(j)}$ denotes the robot contact force corresponding to disturbance $\boldsymbol{w}^{(j)}$. $\xi$ is the disturbance scaling factor, $\zeta$ the minimum total normal force required to maintain an active grasp and $w_\lambda = 0.1$. During each rollout, the CSO in SCSP solves (53) for the nearest points $\boldsymbol{p}_{\text{near}}^i$ to the robot end-effector or Allegro fingers additional to the top 5 pairs of potential grasping pairs, thereby searching for grasping poses online. We use an open-loop baseline that computes the optimal grasping locations offline using CSO and executes them online with CPO for comparison. For bi-manual and multi-finger grasping, we selected five objects each, placed them on a pedestal (black), randomly perturbed their yaw angles $\theta_{\text{yaw}}$ and scaled their sizes by a random factor $\varsigma \in \{0.5, 1, 1.5, 2\}$, conducting 20 trials for each task. The results are shown in Table X and Fig. 17. SCSP achieves a higher success rate than the baseline in these trials; however, it fails on some objects with complex geometry because automatic convexification of the collision mesh by `MuJoCo`.

## B. Case Study II: Articulated Object Manipulation

SCSP uses active contact selection to perform articulated object manipulation. We evaluate this capability on a drawer and a microwave door, which require sequential contact interactions. The tasks are to pull the drawer to a target displacement and to open the microwave door to a target angle. In both tasks, the object is semantically segmented into

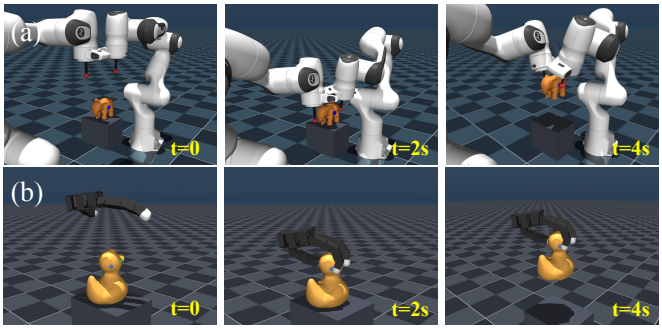



Fig. 17. Snapshots of the multi-contact grasping tasks. (a) Bi-manual grasping of an elephant with the size scaling factor $\varsigma = 1$. (b) Multi-finger grasping of a duck with $\varsigma = 1.5$.

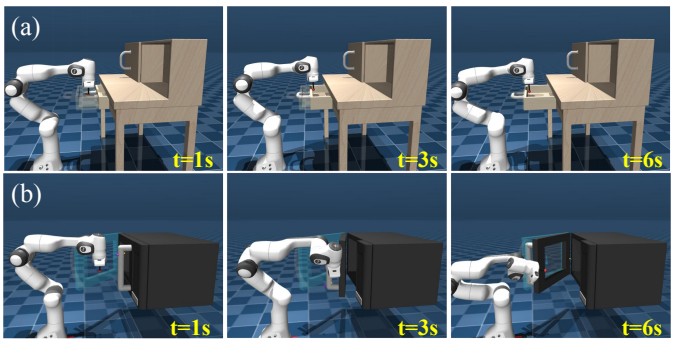



Fig. 18. Snapshots of long-horizon tasks. Given only the target pose of the object, SCSP performs online reasoning and selects appropriate contact locations to accomplish the task. (a) Opening a drawer. (b) Opening the microwave door.

different parts, such as handles, while non-manipulable parts are excluded. For the drawer, we directly compute the final pulled-out pose as the goal pose. For the microwave, which involves rotational motion, we set the goal pose of each rollout as the `SLERP` interpolation between the current state and the desired door state. The parameter and cost settings of SCSP are the same as those in Section VI-A, with $w_{\text{stab}} = 0$. The solver for CPO is an MPPI-based motion planner [51]. As a baseline, we use an open-loop method that plans the contact sequence offline using CSO and executes it online with CPO, in order to demonstrate the effectiveness of online contact selection. We randomly perturbed the distance and yaw angle between the drawer or microwave and the Franka base by $\Delta d \in [-0.1, 0.1]\,\text{m}$ and $\Delta\theta_{\text{yaw}} \in \left[-\frac{\pi}{4}, \frac{\pi}{4}\right]$ and conducted 10 trials for each object. The experimental results are shown in Fig. 18, Table X and the website videos. Although the success rates are comparable, the open-loop method cannot handle certain execution errors, such as contact slippage or the robot approaching singular configurations.

# VIII. Discussion & Limitations

The theoretical and experimental results support the usefulness of explicit contact selection, while also exposing several limitations of the current implementation.

CSO provides contact-location priors through mesh discretization and a lightweight surrogate contact model. This

TABLE X
SUCCESS TRIALS ACROSS DIFFERENT TASKS.

| Metric | Soccer Dribbling | | Tilted Pushing | | Bi-Manual Grasping | | Multi-finger Grasping | | Long-horizon Tasks | |
|---|---|---|---|---|---|---|---|---|---|---|
| | Open-Loop | SCSP | Open-Loop | SCSP | Open-Loop | SCSP | Open-Loop | SCSP | Open-Loop | SCSP |
| **Success Trials** | / | **10/10** | 0/20 | **18/20** | 0/20 | **15/20** | 15/20 | **16/20** | 15/20 | **20/20** |

design is computationally efficient, but its accuracy decreases in complex contact configurations because it assumes a fixed object–environment contact set, neglects inter-contact coupling, and represents contacts as points. CSO therefore does not provide sequential, high-fidelity state prediction and is not directly suited to complex end-effectors or deformable objects. A higher-fidelity contact model, such as the model in [52], could be used when these capabilities are required.

CPO generates trajectories for contact-free and contact-rich stages using model-predictive control. Its current implementation may be less suitable for tasks such as loco-manipulation and may not maintain continuous surface contact during contact switching. Future work can replace the rule-based ranking strategy with a learned representation and can combine CPO with a more expressive motion planner such as [53].

The experiments also suggest that the interface between contact selection and trajectory optimization is important: CPO can use a feasible nearby contact point when the lowest-cost sampled point is difficult to execute. This observation does not establish that the surrogate model is accurate outside the evaluated tasks, and the conclusions about robustness should therefore be interpreted within the tested range of geometries, disturbances, and model mismatch.

## IX. CONCLUSION

This work presented SCSP, a cascaded framework that separates contact-location selection from contact-trajectory optimization for contact-rich manipulation. CSO evaluates a finite set of sampled surface points with a surrogate contact model, and CPO uses the resulting contact prior together with a ranking rule to generate approaching, contact, and contact-switching trajectories. Simulations and hardware experiments show that this decomposition can improve manipulation diversity and task success under the tested geometric and modeling conditions. The results also indicate that SCSP remains limited by the point-contact and fixed-contact-set assumptions of the surrogate model, as well as by the need for a one-time manual MuJoCo calibration in the vision-based hardware experiments.

## REFERENCES


[1] C. Zito, R. Stolkin, M. Kopicki, and J. L. Wyatt, "Two-level RRT planning for robotic push manipulation," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2012, pp. 678–685.

[2] A. Wu, S. Sadraddini, and R. Tedrake, "R3T: Rapidly-exploring random reachable set tree for optimal kinodynamic planning of nonlinear hybrid systems," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2020, pp. 4245–4251.

[3] J. A. Haustein, J. King, S. S. Srinivasa, and T. Asfour, "Kinodynamic randomized rearrangement planning via dynamic transitions between statically stable states," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2015, pp. 3075–3082.

[4] C. Chen, P. Culbertson, M. Lepert, M. Schwager, and J. Bohg, "Trajectotree: Trajectory optimization meets tree search for planning multi-contact dexterous manipulation," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2021, pp. 8262–8268.

[5] X. Cheng, E. Huang, Y. Hou, and M. T. Mason, "Contact mode guided motion planning for quasidynamic dexterous manipulation in 3D," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2022, pp. 2730–2736.

[6] B. Aceituno-Cabezas and A. Rodriguez, "A global quasi-dynamic model for contact-trajectory optimization in manipulation," 2020.

[7] T. Marcucci, R. Deits, M. Gabiccini, A. Bicchi, and R. Tedrake, "Approximate hybrid model predictive control for multi-contact push recovery in complex environments," in *Proc. IEEE-RAS Int. Conf. Humanoid Robots (Humanoids)*, 2017, pp. 31–38.

[8] F. R. Hogan and A. Rodriguez, "Feedback control of the pusher-slider system: A story of hybrid and underactuated contact dynamics," in *Algorithmic Foundations of Robotics XII*, 2020, pp. 800–815.

[9] M. Kim, J. Han, J. Kim, and B. Kim, "Pre- and post-contact policy decomposition for non-prehensile manipulation with zero-shot sim-to-real transfer," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2023, pp. 10 644–10 651.

[10] J. Liang, X. Cheng, and O. Kroemer, "Learning preconditions of hybrid force-velocity controllers for contact-rich manipulation," in *Proc. Conf. Robot Learn. (CoRL)*.

[11] W. Zhou and D. Held, "Learning to grasp the ungraspable with emergent extrinsic dexterity," in *Proc. Conf. Robot Learn. (CoRL)*, 2023, pp. 150–160.

[12] X. Zhang, S. Jain, B. Huang, M. Tomizuka, and D. Romeres, "Learning generalizable pivoting skills," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2023, pp. 5865–5871.

[13] W. Zhou, B. Jiang, F. Yang, C. Paxton, and D. Held, "HACMan: Learning hybrid actor-critic maps for 6D non-prehensile manipulation," in *Proc. Conf. Robot Learn. (CoRL)*, 2023, pp. 241–265.

[14] Y. Cho, J. Han, Y. Cho, and B. Kim, "Corn: Contact-based object representation for nonprehensile manipulation of general unseen objects," *arXiv preprint arXiv:2403.10760*, 2024.

[15] B. Jiang, Y. Wu, W. Zhou, C. Paxton, and D. Held, "Hacman++: Spatially-grounded motion primitives for manipulation," *arXiv preprint arXiv:2407.08585*, 2024.

[16] H. Le, T. Hoang, M. Gabriel, G. Neumann, and N. A. Vien, "Enhancing exploration with diffusion policies in hybrid off-policy RL: Application to non-prehensile manipulation," *IEEE Robot. Autom. Lett.*, 2025.

[17] J. Lyu, Z. Li, X. Shi, C. Xu, Y. Wang, and H. Wang, "Dywa: Dynamics-adaptive world action model for generalizable non-prehensile manipulation," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, 2025, pp. 11 058–11 068.

[18] A. Aydinoglu, A. Wei, W.-C. Huang, and M. Posa, "Consensus complementarity control for multi-contact MPC," *IEEE Trans. Robot.*, 2024.

[19] Y. Li, H. Han, S. Kang, J. Ma, and H. Yang, "On the surprising robustness of sequential convex optimization for contact-implicit motion planning," *arXiv preprint arXiv:2502.01055*, 2025.

[20] V. Kurtz, A. Castro, A. Ö. Önol, and H. Lin, "Inverse dynamics trajectory optimization for contact-implicit model predictive control," *Int. J. Robot. Res.*, vol. 45, no. 1, pp. 23–40, 2026.

[21] T. Pang, H. J. T. Suh, L. Yang, and R. Tedrake, "Global planning for contact-rich manipulation via local smoothing of quasi-dynamic contact models," *IEEE Trans. Robot.*, vol. 39, no. 6, pp. 4691–4711, 2023.

[22] W. Jin, "Complementarity-free multi-contact modeling and optimization for dexterous manipulation," *arXiv preprint arXiv:2408.07855*, 2024.

[23] M. Zhang, D. K. Jha, A. U. Raghunathan, and K. Hauser, "Simultaneous trajectory optimization and contact selection for contact-rich manipulation with high-fidelity geometry," *IEEE Trans. Robot.*, 2025.

[24] W. Yang and W. Jin, "Contactsdf: Signed distance functions as multi-contact models for dexterous manipulation," *arXiv preprint arXiv:2408.09612*, 2024.

[25] S. Venkatesh, B. Bianchini, A. Aydinoglu, W. Yang, and M. Posa, "Approximating global contact-implicit MPC via sampling and local complementarity," *arXiv preprint arXiv:2505.13350*, 2025.
[26] M. Posa, C. Cantu, and R. Tedrake, "A direct method for trajectory optimization of rigid bodies through contact," *Int. J. Robot. Res.*, vol. 33, no. 1, pp. 69–81, 2014.
[27] P. M. Wensing, M. Posa, Y. Hu, A. Escande, N. Mansard, and A. Del Prete, "Optimization-based control for dynamic legged robots," *IEEE Trans. Robot.*, vol. 40, pp. 43–63, 2023.
[28] D. Stewart and J. C. Trinkle, "An implicit time-stepping scheme for rigid body dynamics with Coulomb friction," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, vol. 1, 2000, pp. 162–169.
[29] M. Anitescu, "Optimization-based simulation of nonsmooth rigid multibody dynamics," *Math. Program.*, vol. 105, no. 1, pp. 113–143, 2006.
[30] H. Bui, Y. Gao, H. Yang, E. Cui, S. Mody, B. Acosta, T. S. Felix, B. Bianchini, and M. Posa, "Push anything: Single- and multi-object pushing from first sight with contact-implicit MPC," *arXiv preprint arXiv:2510.19974*, 2025.
[31] H. Suh, T. Pang, T. Zhao, and R. Tedrake, "Dexterous contact-rich manipulation via the contact trust region," *arXiv preprint arXiv:2505.02291*, 2025.
[32] Y. Jiang, M. Yu, X. Zhu, M. Tomizuka, and X. Li, "Robust model-based in-hand manipulation with integrated real-time motion-contact planning and tracking," *arXiv preprint arXiv:2505.04978*, 2025.
[33] Z. Xie, Y. Xiang, M. Posa, and W. Jin, "Where to touch, how to contact: Hierarchical RL-MPC framework for geometry-aware long-horizon dexterous manipulation," *arXiv preprint arXiv:2601.10930*, 2026.
[34] T. Pang and R. Tedrake, "A convex quasistatic time-stepping scheme for rigid multibody systems with contact and friction," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2021, pp. 6614–6620.
[35] N. Hogan, "Impedance control: An approach to manipulation," in *Proc. Amer. Control Conf. (ACC)*, 1984, pp. 304–313.
[36] M. Anitescu and A. Tasora, "An iterative approach for cone complementarity problems for nonsmooth dynamics," *Comput. Optim. Appl.*, vol. 47, no. 2, pp. 207–235, 2010.
[37] M. Avriel, *Nonlinear programming: Analysis and methods*. Courier Corporation, 2003.
[38] J.-M. Lien and N. M. Amato, "Approximate convex decomposition of polyhedra," in *Proc. ACM Symp. Solid Phys. Model.*, 2007, pp. 121–131.
[39] B. Acosta and M. Posa, "Perceptive mixed-integer footstep control for underactuated bipedal walking on rough terrain," *IEEE Trans. Robot.*, 2025.
[40] J. Nakanishi, R. Cory, M. Mistry, J. Peters, and S. Schaal, "Operational space control: A theoretical and empirical comparison," *Int. J. Robot. Res.*, vol. 27, no. 6, pp. 737–757, 2008.
[41] G. Williams, A. Aldrich, and E. Theodorou, "Model predictive path integral control using covariance variable importance sampling," *arXiv preprint arXiv:1509.01149*, 2015.
[42] S. Brahmbhatt, C. Ham, C. C. Kemp, and J. Hays, "Contactdb: Analyzing and predicting grasp contact via thermal imaging," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2019, pp. 8709–8719.
[43] E. Todorov, T. Erez, and Y. Tassa, "MuJoCo: A physics engine for model-based control," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst. (IROS)*, 2012, pp. 5026–5033.
[44] V. Makoviychuk, L. Wawrzyniak, Y. Guo, M. Lu, K. Storey, M. Macklin, D. Hoeller, N. Rudin, A. Allshire, A. Handa, and G. State, "Isaac Gym: High performance GPU-based physics simulation for robot learning," arXiv preprint arXiv:2108.10470, 2021.
[45] X. Wei, M. Liu, Z. Ling, and H. Su, "Approximate convex decomposition for 3D meshes with collision-aware concavity and tree search," *ACM Trans. Graph.*, vol. 41, no. 4, pp. 1–18, 2022.
[46] B. Wen, W. Yang, J. Kautz, and S. Birchfield, "Foundationpose: Unified 6d pose estimation and tracking of novel objects," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2024, pp. 17 868–17 879.
[47] S. Yin, Y. Ze, H.-X. Yu, C. K. Liu, and J. Wu, "Visualmimic: Visual humanoid loco-manipulation via motion tracking and generation," *arXiv preprint arXiv:2509.20322*, 2025.
[48] J. Kong, X. Liu, Y. Lin, J. Han, S. Schwertfeger, C. Bai, and X. Li, "Learning soccer skills for humanoid robots: A progressive perception-action framework," *arXiv preprint arXiv:2602.05310*, 2026.
[49] M. Lin, Y.-L. Wei, J. Chen, Y. Lin, S. Chen, J. Lyu, J. Chen, Y. Tang, H. Wang, and W.-S. Zheng, "Bidexgrasp: Coordinated bimanual dexterous grasps across object geometries and sizes," *arXiv preprint arXiv:2604.06589*, 2026.
[50] C. Borst, M. Fischer, and G. Hirzinger, "Grasp planning: How to choose a suitable task wrench space," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, vol. 1, 2004, pp. 319–325.
[51] M. Bhardwaj, B. Sundaralingam, A. Mousavian, N. D. Ratliff, D. Fox, F. Ramos, and B. Boots, "Storm: An integrated framework for fast joint-space model-predictive control for reactive manipulation," in *Proc. Conf. Robot Learn. (CoRL)*, 2022, pp. 750–759.
[52] W. Huang, Y.-W. Chao, A. Mousavian, M.-Y. Liu, D. Fox, K. Mo, and L. Fei-Fei, "Pointworld: Scaling 3D world models for in-the-wild robotic manipulation," *arXiv preprint arXiv:2601.03782*, 2026.
[53] S. Xu, S. Schulter, M. Ziyadi, X. He, X. Fei, Y.-X. Wang, and L. Gui, "Interprior: Scaling generative control for physics-based human-object interactions," *arXiv preprint arXiv:2602.06035*, 2026.

## APPENDIX

### A. Proof of Closest Approximation

Let $\boldsymbol{A}_{ii} = \boldsymbol{W}_{ii} - \boldsymbol{R}_{ii} \in \mathbb{R}^{n\times n}$ be the unregularized contact mobility matrix. We seek a diagonal matrix $D = \operatorname{diag}(d_1, \ldots, d_n)$ that minimizes the Frobenius norm of the approximation error:

$$\boldsymbol{D}^* = \arg\min_{\boldsymbol{D}} \|\boldsymbol{A}_{ii} - \boldsymbol{D}\|_{\mathrm{F}}^2 . \tag{54}$$

Expanding the Frobenius norm, we have

$$\|\boldsymbol{A}_{ii} - \boldsymbol{D}\|_{\mathrm{F}}^2 = \sum_{j\neq k} |(\boldsymbol{A}_{ii})_{jk}|^2 + \sum_{j=1}^{n} |(\boldsymbol{A}_{ii})_{jj} - d_j|^2. \tag{55}$$

The off-diagonal terms are not affected by choosing $D$, and so the minimization reduces to independent scalar problems for each diagonal entry:

$$d_j^* = \arg\min_{d_j} |(\boldsymbol{A}_{ii})_{jj} - d_j|^2, \quad j = 1, \ldots, n. \tag{56}$$

It is immediate that the minimizer is $d_j^* = (\boldsymbol{A}_{ii})_{jj}$. Hence $\operatorname{diag}(\boldsymbol{A}_{ii})$ is the closest diagonal approximation in the Frobenius norm. Equation (27) adds $r\boldsymbol{I}$, matching the regularization in the coupled reference.